

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему
**СЕТЕВАЯ РАСПРЕДЕЛЕННАЯ ОЧЕРЕДЬ СООБЩЕНИЙ, ДОСТУПНАЯ
КЛИЕНТАМ ПО СТАНДАРТНОМУ ПРОТОКОЛУ HTTP**

БГУИР КП 6-05 06 12 01 026 ПЗ

Студент

Е. В. Лобач

Руководитель

С. В. Болтак

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	7
1.1 Сравнительный анализ существующих решений	7
1.2 Постановка задачи.....	8
2 Проектирование программы	10
2.1 Проектирование структуры программы	10
2.2 Проектирование системы сетевого взаимодействия	12
3 Программная реализация	14
3.1 Описание разработанных модулей	14
3.2 Описание сетевого модуля	15
4 Тестирование	17
5 Руководство пользователя	19
5.1 Спецификация REST API	19
5.2 Описание графического интерфейса панели управления	20
5.3 Пример типового сценария работы с системой	23
Заключение	32
Список использованных источников.....	33
Приложение А (обязательное) Листинг кода с комментариями.....	34
Приложение Б (обязательное) Схема алгоритма работы системы.....	58

ВВЕДЕНИЕ

В эпоху цифровизации и повсеместного перехода к микросервисной архитектуре одним из ключевых требований к любой распределённой системе становится обеспечение надёжного, отказоустойчивого и масштабируемого асинхронного взаимодействия между программными модулями [1]. Традиционная модель синхронного взаимодействия на основе протокола *HTTP*, реализуемая по парадигме «запрос – ответ», обладает рядом существенных ограничений. При пиковых нагрузках или сбоях в работе смежных сервисов синхронные вызовы способны вызывать каскадные отказы всей системы, исчерпание пула потоков веб-сервера и значительную деградацию качества обслуживания пользователей.

Одним из наиболее эффективных архитектурных решений данной проблемы является применение брокеров сообщений – специализированных инфраструктурных систем, управляющих маршрутизацией, временным хранением и гарантированной доставкой данных между сервисами-отправителями и сервисами-получателями. Брокеры сообщений обеспечивают слабую связность компонентов, позволяют сглаживать пиковые нагрузки за счёт буферизации и предоставляют встроенные механизмы повторной обработки задач в случае временных сбоев. Благодаря этим свойствам брокеры сообщений стали неотъемлемым элементом архитектуры современных высоконагруженных систем: они применяются в платёжных платформах, системах обработки событий, конвейерах машинного обучения и любых сценариях, где потеря или дублирование задачи недопустимы [2].

Разработка веб-приложения, реализующего функции брокера сообщений, представляет собой комплексную инженерную задачу, находящуюся на стыке проектирования распределённых систем, реализации сетевых протоколов реального времени, построения высокопроизводительных механизмов хранения данных в оперативной памяти и проектирования надёжных реляционных схем для долговременного персистентного хранения. Актуальность работы определяется высоким спросом на лёгкие, глубоко интегрируемые платформы, способные обеспечивать гарантированную доставку сообщений с поддержкой приоритетов, отложенного выполнения и мониторинга в реальном времени – без избыточной сложности, присущей промышленным решениям, таким как *Apache Kafka* или *RabbitMQ* [3].

Существующие промышленные решения, несмотря на свою функциональную полноту, обладают рядом характеристик, делающих их избыточными для широкого класса задач. *Apache Kafka* ориентирована на обработку потоков событий с сохранением полного лога и требует развёртывания кластера координации на базе *ZooKeeper* или *KRaft*, что существенно усложняет инфраструктуру. *RabbitMQ*, реализующий протокол *AMQP*, предоставляет богатую модель маршрутизации, однако его конфигурирование и сопровождение требуют значительных операционных затрат. В условиях, когда команда разрабатывает монолитное

или слабо распределённое приложение на РНР и нуждается в надёжной очереди задач с визуальным мониторингом, развёртывание подобных систем экономически и технически нецелесообразно. Разрабатываемый комплекс закрывает именно эту нишу: он предоставляет достаточный для большинства production-сценариев функциональный набор при минимальных требованиях к инфраструктуре.

Дополнительным фактором, определяющим актуальность работы, является образовательная ценность реализации подобной системы с нуля. Проектирование брокера сообщений требует глубокого понимания атомарных операций в *Redis*, управления транзакциями в *PostgreSQL*, механизмов блокировки и таймаутов видимости, а также организации двунаправленного взаимодействия через *WebSocket*. Совокупность этих задач формирует полноценный инженерный опыт в области построения надёжных распределённых систем, применимый далеко за пределами конкретного проекта.

Объектом исследования являются технологии построения систем управления транзакционными очередями сообщений и механизмы асинхронного межсервисного взаимодействия в современных веб-комплексах.

Предметом исследования выступает процесс проектирования, программной реализации и автоматизированного тестирования сервиса брокера сообщений с использованием технологического стека на основе языка программирования *PHP 8+*, фреймворка *Laravel 11*, реляционной базы данных *PostgreSQL*, *in-memory* хранилища *Redis*, а также сервера веб-сокетов *Laravel Reverb* для широковещательной ретрансляции событий.

Целью курсового проекта является создание надёжной высокопроизводительной программной системы, предоставляющей внешним сервисам унифицированный *REST API* для управления логическими очередями, публикации и конкурентного потребления сообщений, а также визуальный интерфейс для непрерывного мониторинга метрик и состояния брокера в режиме реального времени.

Для достижения поставленной цели необходимо решить следующие задачи: провести сравнительный анализ существующих решений на рынке систем управления очередями; сформировать комплекс функциональных и нефункциональных требований к разрабатываемой системе; спроектировать модульную архитектуру программного комплекса на базе паттерна *MVC* с выделением слоя изолированной бизнес-логики; разработать схему реляционной базы данных для персистентного хранения истории задач; спроектировать алгоритмы маршрутизации и блокировки задач в оперативной памяти с использованием атомарных *Lua*-скриптов в *Redis*; реализовать полнодуплексное взаимодействие для панели мониторинга на основе протокола *WebSocket*; провести автоматизированное тестирование бизнес-логики; разработать техническую документацию и спецификацию *REST API*.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Приступая к проектированию архитектуры отказоустойчивой системы управления очередями сообщений, необходимо провести детальный анализ существующих на рынке программных продуктов. Индустрия разработки программного обеспечения предлагает широкий спектр инструментов – от лёгких очередей задач до масштабных распределённых платформ потоковой передачи данных, каждая из которых ориентирована на определённый класс задач и обладает своим набором компромиссов между производительностью, надёжностью и сложностью эксплуатации.

Среди наиболее распространённых решений, представленных в различных сегментах отрасли, можно выделить четыре продукта, приведённых в таблице 1.1 [4].

Таблица 1.1 – Сравнительный анализ существующих решений

Название продукта	Основной протокол	Механизм хранения	Преимущества	Недостатки
<i>RabbitMQ</i>	<i>AMQP</i>	Диск / <i>RAM</i> (<i>Erlang</i>)	Мощный механизм маршрутизации (<i>exchanges</i> , <i>bindings</i>), высокая надёжность, <i>ACK/NACK</i> из коробки	Сложность настройки кластера, высокие требования к ресурсам, избыточность протокола <i>AMQP</i> для простых задач
<i>Apache Kafka</i>	<i>TCP</i> (<i>Custom</i>)	Диск (<i>Append-only logs</i>)	Колоссальная пропускная способность, распределённая архитектура, длительное хранение потоков данных	Отсутствие классической маршрутизации очередей, сложность поддержки (<i>Zookeeper/KRaft</i>), не подходит для простых <i>task queues</i>
<i>Redis</i> (<i>Pub/Sub & Streams</i>)	<i>RESP</i>	<i>RAM</i> (<i>In-memory</i>)	Сверхвысокая скорость, минимальная задержка, лёгковесность	Отсутствует персистентность при сбоях; повторы и <i>DLQ</i> нужно реализовывать вручную.

Название продукта	Основной протокол	Механизм хранения	Преимущества	Недостатки
Beanstalkd	<i>TCP (Text-based)</i>	<i>RAM + WAL</i>	Предельная простота, встроенная поддержка отложенных задач	Проект практически не развивается, скудный функционал мониторинга, отсутствие репликации и кластеризации

Проведённый анализ показывает, что на рынке преобладают либо тяжелые промышленные решения, такие как *Kafka* и *RabbitMQ*, требующие выделенных команд специалистов для развёртывания и поддержки, либо низкоуровневые инструменты вроде чистого *Redis*, предполагающие значительный объём программной доработки со стороны команды разработчиков. Промежуточная ниша лёгких, легко интегрируемых брокеров с готовым *API* и встроенным мониторингом на практике остаётся незаполненной.

Разрабатываемое веб-приложение призвано занять именно эту нишу. Оно сочетает высокую скорость *Redis* для оперативной маршрутизации и выполнения транзакционных блокировок с надёжностью *PostgreSQL* для гарантированного долговременного хранения истории задач и статистических данных. При этом взаимодействие с системой осуществляется через интуитивно понятный *REST API*, реализованный на базе фреймворка *Laravel*, что существенно снижает порог вхождения и упрощает интеграцию в уже существующие программные комплексы.

1.2 Постановка задачи

На основании проведённого аналитического обзора предметной области сформулировано техническое задание на разработку системы управления очередями сообщений. Проектируемая система должна устранить выявленные недостатки существующих решений, предоставив разработчикам лёгкий в интеграции инструмент с предсказуемым поведением, встроенным мониторингом и чётко определённой семантикой доставки сообщений.

Система представляет собой клиент-серверный веб-комплекс, включающий два взаимодополняющих интерфейса: *REST API* для программного межсервисного взаимодействия и браузерную панель управления, отображающую состояние брокера в режиме реального времени. Взаимодействие между этими компонентами строится на разделении обязанностей: *API* обслуживает внешних потребителей и издателей, тогда как панель мониторинга ориентирована на операционный контроль со стороны команды разработки и эксплуатации.

С точки зрения функциональных требований система охватывает полный жизненный цикл сообщения – от момента публикации до финального подтверждения или перемещения в очередь недоставленных сообщений. В части управления очередями система должна поддерживать создание логических очередей с валидацией уникальности имени, переключение их статуса между активным и приостановленным состоянием, а также удаление с защитой от случайной потери необработанных задач. При постановке очереди на паузу приём новых сообщений продолжается, однако выдача задач потребителям приостанавливается – это позволяет управлять нагрузкой без потери данных.

Жизненный цикл сообщения реализуется через четыре ключевые операции. При публикации система принимает произвольную полезную нагрузку в формате *JSON*, позволяет задать один из трёх уровней приоритета (высокий, нормальный, низкий), указать задержку перед доставкой в секундах и ограничить максимальное количество попыток обработки. При потреблении задача выдаётся атомарно, исключая состояние гонки между конкурирующими потребителями; одновременно устанавливается таймаут видимости и генерируется криптографический токен подтверждения. Подтверждение успешной обработки переводит сообщение в финальный статус и сохраняет результат, возвращённый потребителем. В случае отказа система инкрементирует счётчик попыток и либо возвращает задачу в очередь готовности, если лимит не исчерпан, либо перемещает её в *Dead Letter Queue* со статусом «*dead*».

Отдельного внимания заслуживает механизм восстановления, реализованный в виде фонового процесса. Он периодически сканирует задачи, у которых истёк таймаут видимости, а также отложенные сообщения, время доставки которых уже наступило, и повторно активизирует их. Именно этот механизм обеспечивает устойчивость системы к сбоям на стороне потребителей – зависшая задача не теряется, а возвращается в обработку автоматически. В контексте маршрутов проекта это соответствует эндпоинту */broker/recover*, реализованному в *BrokerController*.

Панель мониторинга обеспечивает визуализацию ключевых метрик брокера в реальном времени: распределение сообщений по статусам, состояние отдельных очередей и журнал последних событий. Обновление данных осуществляется через протокол *WebSocket*, что исключает необходимость периодического опроса сервера и снижает задержку отображения изменений.

Нефункциональные требования устанавливают конкретные ориентиры по производительности и консистентности. Время ответа на операции публикации и потребления не должно превышать 20–30 миллисекунд, что достигается за счёт использования *Redis* в качестве основного маршрутизатора задач – все операции критического пути выполняются в оперативной памяти. Консистентность между *Redis* и *PostgreSQL* обеспечивается на уровне сервисного слоя приложения посредством транзакций базы данных: состояние задачи изменяется атомарно в обоих хранилищах, что исключает рассогласование при частичных сбоях.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

2.1 Проектирование структуры программы

Архитектура разрабатываемой программной системы построена на принципах паттерна *MVC (Model-View-Controller)* с введением дополнительного слоя сервисов [5]. Проект реализован на базе фреймворка *Laravel 11*, который предоставляет развитую инфраструктуру для построения веб-приложений: систему маршрутизации, *ORM*-слой *Eloquent*, механизм валидации запросов и встроенные средства работы с очередями [6]. Использование выделенного сервисного слоя позволяет полностью отделить бизнес-логику брокера от *HTTP*-контроллеров, что делает код независимым от транспортного уровня, удобным для автоматизированного тестирования и пригодным для повторного использования в различных контекстах.

Внутренняя структура серверного приложения включает несколько уровней абстракции, каждый из которых несёт строго определённую ответственность.

Уровень маршрутизации и *API*-контроллеров реализован в файле *api.php* и включает три контроллера: *QueueController*, *MessageController* и *BrokerController*. Контроллеры не содержат бизнес-логики – их единственная обязанность состоит в приёме *HTTP*-запроса, передаче данных в соответствующий валидатор на основе *FormRequest* и формировании ответа через *JsonResource*. Такое разделение обязанностей обеспечивает тонкость контроллеров и концентрирует всю логику в нижележащих слоях.

Сервисный слой представляет собой ядро системы и включает пять специализированных служб. *QueueService* управляет жизненным циклом очередей: обрабатывает их создание, изменение статуса, удаление и сбор агрегированной статистики, объединяя данные из *PostgreSQL* и *Redis* в единое представление. *MessagePublishService* реализует транзакционную логику публикации: сообщение атомарно записывается в реляционную базу данных, после чего его идентификатор помещается в соответствующую структуру *Redis* – в сортированное множество для отложенных задач или в список для задач, готовых к немедленной выдаче. *MessageConsumeService* инкапсулирует вызов атомарных *Lua*-скриптов для извлечения задачи из очереди, генерирует уникальный токен подтверждения, хэширует его перед сохранением и возвращает оригинальное значение потребителю. *MessageAckService* обрабатывает как положительные, так и отрицательные ответы воркеров: при успешном подтверждении переводит задачу в финальный статус и сохраняет результат, при отказе – инкрементирует счётчик попыток и либо возвращает задачу в очередь, либо перемещает её в *Dead Letter Queue*. Наконец, *MessageRecoveryService* выполняет роль фонового сборщика: периодически сканирует задачи с истёкшим таймаутом видимости и отложенные сообщения с наступившим временем доставки, возвращая их в активный пул обработки.

Уровень интеграции с Redis реализован в виде отдельного компонента *RedisBroker*, абстрагирующего низкоуровневые команды хранилища от остальных частей системы. Для управления готовыми к выдаче задачами используются списки с операциями *LPUSH* и *RPOP*, обеспечивающими порядок *FIFO* в рамках одного уровня приоритета. Отложенные задачи хранятся в сортированных множествах (*ZADD*, *ZREM*, *ZRANGEBYSCORE*), где в качестве оценки используется временная метка доступности, что позволяет эффективно извлекать задачи, готовые к активации, за время $O(\log N)$ [7].

Основой надёжности системы с точки зрения долговременного хранения данных является реляционная база данных *PostgreSQL*. Схема базы данных включает две основные сущности: *queues* и *messages*. Таблица *queues* хранит метаданные логических очередей, тогда как таблица *messages* содержит полную информацию о каждом сообщении на протяжении всего его жизненного цикла. Структура таблицы *messages* приведена в таблице 2.1.

Таблица 2.1 – Структура таблицы *messages* (*PostgreSQL*)

Поле	Тип данных	Описание и назначение
<i>payload</i>	<i>JSONB</i>	Полезная нагрузка задачи, передаваемая потребителю
<i>status</i>	<i>VARCHAR</i>	Статус сообщения: <i>pending</i> , <i>delayed</i> , <i>processing</i> , <i>done</i> , <i>failed</i> , <i>dead</i>
<i>attempts</i> / <i>max_attempts</i>	<i>SMALLINT</i>	Текущее и максимальное количество попыток выполнения
<i>available_at</i>	<i>TIMESTAMP</i>	Момент времени, начиная с которого задача доступна для выдачи
<i>reserved_until</i>	<i>TIMESTAMP</i>	Срок действия блокировки в рамках таймаута видимости
<i>consumer_id</i>	<i>VARCHAR</i>	Идентификатор воркера, взявшего задачу в обработку
<i>ack_token_hash</i>	<i>VARCHAR</i>	<i>SHA</i> -256 хэш токена для верификации завершения задачи
<i>result</i> / <i>error</i>	<i>JSONB</i> / <i>TEXT</i>	Данные или текст ошибки, возвращаемые воркером при завершении

Выбор типа *JSONB* для полей *payload* и *result* обусловлен тем, что *PostgreSQL* хранит данные в этом формате в бинарном виде, что обеспечивает эффективную индексацию и возможность выполнения запросов непосредственно к содержимому *JSON*-документа без его повторного разбора [8]. Поле *ack_token_hash* намеренно хранит не оригинальный токен, а его хэш: это исключает возможность несанкционированного подтверждения задачи даже в случае компрометации базы данных.

2.2 Проектирование системы сетевого взаимодействия

Система коммуникации брокера разделена на два независимых вектора: *RESTful API* для синхронного обмена данными между микросервисами и протокол *WebSocket* для обеспечения мониторинга в режиме реального времени [9]. Такое разделение позволяет оптимизировать каждый канал под его конкретную задачу, не создавая излишней нагрузки на компоненты, не предназначенные для данного типа взаимодействия.

RESTful API обслуживает основной поток операций брокера: публикацию сообщений со стороны издателей и потребление задач со стороны воркеров. Все запросы передают данные в формате *JSON*, а маршрутизация реализована через единую точку входа в соответствии со структурой, определённой в файле *api.php*. Ключевой задачей этого слоя является обеспечение минимальной задержки при высоком уровне параллелизма. Прямое использование *PostgreSQL* в качестве основной очереди потребовало бы интенсивного применения блокировок на уровне строк (*SELECT FOR UPDATE SKIP LOCKED*), что при большом числе одновременно работающих воркеров приводит к существенной деградации производительности из-за конкуренции за блокировки и роста числа ожидающих транзакций.

Для решения этой проблемы была спроектирована гибридная архитектура хранения. *PostgreSQL* выполняет роль единственного источника истины: в нём персистентно хранится полная история сообщений со всеми метаданными, и именно эта база данных является авторитетным источником при восстановлении после сбоев. Физическая же очередь задач, по которой осуществляется маршрутизация в реальном времени, строится в оперативной памяти *Redis*. Операции над ней выполняются на порядки быстрее дисковых и не требуют управления транзакционными блокировками в реляционной базе. Синхронизация между двумя хранилищами обеспечивается на уровне сервисного слоя: каждая операция, изменяющая состояние сообщения, выполняется в рамках транзакции *PostgreSQL*, и лишь после её успешной фиксации вносятся соответствующие изменения в структуры *Redis*. Такой порядок гарантирует, что при частичном сбое реляционная база всегда остаётся консистентным источником для последующего восстановления.

Структуры данных *Redis* спроектированы с учётом требований к приоритетности и отслеживанию состояния задач. Для каждой очереди и каждого уровня приоритета создаётся отдельный список: например, задачи с высоким приоритетом хранятся в ключе *broker:queue:jobs:ready:high*. Воркер при потреблении последовательно обращается сначала к списку высокого приоритета, затем к нормальному и низкому, что реализует вытесняющую приоритетную очередь без дополнительной логики сортировки. Задачи, находящиеся в процессе обработки, перемещаются в отдельное отсортированное множе-

ство `broker:queue:jobs:processing`, где в качестве оценки каждого элемента используется Unix-метка времени истечения таймаута видимости. Это позволяет службе восстановления эффективно находить зависшие задачи единственным запросом `ZRANGEBYSCORE` с верхней границей, равной текущему времени.

Для обеспечения атомарности операций потребления, при которых необходимо одновременно извлечь задачу из списка готовых и поместить её в множество обрабатываемых, применяются *Lua*-скрипты, исполняемые на стороне *Redis*. Поскольку *Redis* выполняет скрипты как единую атомарную операцию, подобный подход полностью исключает состояние гонки между конкурирующими воркерами без использования распределённых блокировок. Применение *Lua* также позволяет объединить несколько логически связанных команд в одно сетевое обращение, что дополнительно сокращает задержку на операциях критического пути. Скрипты компилируются и кэшируются сервером *Redis* при первом вызове посредством команды *EVALSHA*, что исключает накладные расходы на повторную передачу тела скрипта при каждом запросе и обеспечивает стабильное время выполнения вне зависимости от размера логики [10].

Второй вектор коммуникации – протокол *WebSocket* – обеспечивает работу интерактивной панели мониторинга. Для его реализации применяется система событийного вещания *Laravel Event Broadcasting* в связке с сервером *Laravel Reverb*. *Reverb* представляет собой современный встроенный *WebSocket*-сервер, реализованный на *PHP* с использованием асинхронной модели ввода-вывода, что позволяет ему обслуживать большое число одновременных соединений без создания отдельного потока на каждое подключение. Это делает его производительной и архитектурно однородной альтернативой решениям на базе *Node.js*.

Механизм распространения событий устроен следующим образом. При любом изменении состояния брокера – публикации новой задачи, её успешном завершении, отказе или изменении статуса очереди – сервисный слой генерирует событие *BrokerStateUpdated*. Это событие сериализует текущий снимок статистики брокера и передаёт его по *TCP* на сервер *Reverb*. *Reverb* рассылает полученный *JSON*-пакет по протоколу *WebSocket* всем административным клиентам, подключённым к каналу *broker*. На стороне браузера библиотека *Laravel Echo* перехватывает событие *.broker.state.updated* и динамически обновляет интерфейс панели управления без перезагрузки страницы. Такая архитектура гарантирует, что задержка между изменением состояния системы и его отображением в интерфейсе определяется только сетевыми характеристиками и не зависит от интервала опроса.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Описание разработанных модулей

Программный комплекс реализован в виде набора строго типизированных сервисных классов, каждый из которых несёт изолированную ответственность за определённый аспект жизненного цикла сообщения. Наиболее сложным и критически важным с алгоритмической точки зрения является модуль конкурентного извлечения задач, реализованный в классе *MessageConsumeService* и опирающийся на низкоуровневый компонент *RedisBroker*.

Проблема конкурентного доступа возникает в ситуации, когда несколько воркеров одновременно обращаются к одной очереди за следующей задачей. При наивной реализации, основанной на последовательном чтении и удалении элемента из списка, два воркера могут извлечь один и тот же идентификатор сообщения в промежутке между операциями чтения и удаления. Использование стандартной атомарной команды *Redis RPOP* решает проблему двойного извлечения, однако не устраняет смежную задачу: извлечённое сообщение должно не просто исчезнуть из очереди готовых, но и немедленно появиться в множестве обрабатываемых задач. Если воркер аварийно завершит работу после извлечения, но до регистрации задачи в списке *processing*, сообщение будет безвозвратно потеряно – оно не будет существовать ни в одной из структур данных.

Эта проблема решена путём применения атомарного *Lua*-скрипта, исполняемого непосредственно внутри движка *Redis*. Поскольку *Redis* выполняет *Lua*-скрипты как единую неделимую операцию, никакая внешняя команда не может быть вставлена между её шагами, что полностью исключает состояние гонки. Скрипт принимает ключи списков готовности для каждого уровня приоритета и ключ сортированного множества обрабатываемых задач, после чего последовательно перебирает очереди от высокого приоритета к низкому. При обнаружении первого доступного идентификатора сообщения скрипт атомарно извлекает его из списка и немедленно помещает в множество *processing*, используя в качестве оценки *Unix*-метку времени истечения таймаута видимости. Таким образом, задача в любой момент времени гарантированно присутствует ровно в одной структуре данных *Redis*, что делает систему устойчивой к сбоям на стороне воркеров.

После успешного извлечения идентификатора из *Redis* класс *MessageConsumeService* формирует криптографически стойкий токен подтверждения. Важной деталью реализации является то, что в базу данных *PostgreSQL* сохраняется не сам токен, а его *SHA-256* хэш. Этот подход аналогичен практике хранения хэшей паролей: даже при полной компрометации базы данных злоумышленник не сможет восстановить оригинальный токен и использовать его для несанкционированного подтверждения чужих задач. Оригинальный токен при этом кэшируется в *Redis* с временем жизни, равным сумме тай-

маута видимости и дополнительных 60 секунд. Этот буфер обеспечивает корректную обработку граничных случаев, когда воркер завершает работу в самый последний момент перед истечением таймаута.

Модуль *MessageAckService* реализует строгую двухэтапную проверку входящих запросов на подтверждение. На первом этапе проверяется наличие токена в *Redis*: если запись уже истекла, значит таймаут видимости исчерпан и задача могла быть повторно выдана другому воркеру, поэтому подтверждение отклоняется. На втором этапе вычисляется *SHA-256* хэш предъявленного токена и сравнивается с хэшем, хранящимся в *PostgreSQL*. Только при одновременном выполнении обоих условий система считает запрос легитимным и приступает к финализации задачи. После успешной валидации задача транзакционно переводится в статус *done* в базе данных, сохраняется результат, возвращённый воркером, а метаданные задачи удаляются из множества обрабатываемых задач в *Redis*. Транзакционность этой операции критически важна: частичное обновление, при котором статус изменился в *PostgreSQL*, но запись в *Redis* осталась, привело бы к повторной активации уже завершённой задачи службой восстановления, что нарушило бы гарантии однократной доставки.

3.2 Описание сетевого модуля

Сетевая инфраструктура проекта конфигурируется двумя ключевыми файлами: *config/reverb.php* и *config/broadcasting.php*. Первый определяет параметры запуска самого *WebSocket*-сервера – в частности, порт прослушивания (по умолчанию 8080), хост привязки и параметры допустимых подключений, включая перечень разрешённых источников (*CORS*). Второй задаёт драйвер вещания для всего приложения *Laravel*, указывая фреймворку использовать *Reverb* в качестве транспорта для всех широковещательных событий. Такое разделение конфигурации позволяет независимо управлять параметрами сервера и параметрами системы вещания, не затрагивая остальные части приложения. *Laravel Reverb* функционирует как самостоятельный долгоживущий процесс, запускаемый командой *php artisan reverb:start* параллельно с основным *HTTP*-сервером, удерживает постоянные *TCP*-соединения с подключёнными браузерами и поддерживает совместимый с *Pusher* протокол на уровне *WebSocket*, что позволяет использовать стандартную клиентскую библиотеку *Laravel Echo* без каких-либо модификаций.

При наступлении любого значимого изменения в системе – создания или удаления очереди, публикации, потребления, подтверждения или отклонения сообщения – сервисный слой инициирует событие *App\Events\BrokerStateUpdated*. Вызов *event(new BrokerStateUpdated(\$queue->name))* присутствует во всех соответствующих методах сервисного слоя, что обеспечивает полное покрытие жизненного цикла брокера: ни одно изменение состояния не остаётся без оповещения подключённых клиентов. Ключевой особенностью класса *BrokerStateUpdated* является реализация интерфейса *ShouldBroadcastNow* [11], который сигнализирует

ядру Laravel о необходимости выполнить сериализацию и отправку события на WebSocket-сервер немедленно, в рамках текущего процесса обработки HTTP-запроса, минуя стандартную систему фоновых очередей фреймворка. Это архитектурное решение принципиально важно для минимизации задержки: при использовании обычного интерфейса `ShouldBroadcast` обновление откладывалось бы до момента, когда воркер очереди заберёт задачу на обработку, что вносило бы непредсказуемую и неприемлемую для мониторинга в реальном времени задержку.

Событие публикуется в публичный канал *broker* под именем *broker.state.updated*. Выбор публичного канала обусловлен тем, что дашборд брокера является инструментом внутреннего администрирования и не требует механизма авторизации на уровне отдельных пользователей. Вместе с событием передаётся глубоко вложенный *JSON*-объект, формируемый методом *broadcastWith*. Этот объект содержит агрегированную статистику по всем очередям в разрезе статусов сообщений – *pending*, *delayed*, *processing*, *done*, *failed*, *dead* – а также полный список очередей и массив последних сообщений активной очереди (не более 25 записей). Такая структура *payload* намеренно избыточна: клиент получает достаточно данных для полного обновления состояния интерфейса по одному входящему пакету без необходимости выполнять дополнительные запросы к *REST API*, что устраняет риск состояния гонки между двумя последовательными изменениями состояния.

На стороне браузера клиентское приложение использует объект *window.Echo* для подписки на канал *broker* и регистрации функции обратного вызова на событие *.broker.state.updated*. Соединение устанавливается единожды при монтировании корневого компонента и сохраняется на протяжении всей пользовательской сессии. При получении пакета библиотека *Echo* автоматически десериализует *JSON-payload* и передаёт его в зарегистрированный обработчик, который атомарно обновляет реактивное состояние интерфейса: панель статистики, список очередей и таблица сообщений перерисовываются одновременно на основании одного входящего объекта. Таким образом, весь путь от изменения состояния на сервере до обновления *DOM* в браузере выполняется без какого-либо опроса (*polling*) со стороны клиента и определяется исключительно сетевой задержкой между сервером *Reverb* и браузером пользователя.

4 ТЕСТИРОВАНИЕ

Тестирование является критически важным этапом разработки, обеспечивающим корректность работы системы и её соответствие установленным функциональным и нефункциональным требованиям. В рамках разработки программного комплекса было проведено комплексное многоуровневое тестирование, охватывающее проверку механизмов управления очередями, консистентности данных между *PostgreSQL* и *Redis*, а также надёжности протокола подтверждения обработки сообщений. Необходимость такого тестирования обусловлена распределённой природой брокера: корректность системы определяется не только поведением отдельных компонентов, но и строгой согласованностью между реляционной базой данных и кэшем *Redis* при любых сценариях отказа.

Основной целью испытаний является подтверждение готовности системы к эксплуатации. Для этого были решены следующие задачи: проверка функциональности всех модулей – управления очередями, публикации и потребления сообщений, механизмов подтверждения (*ack/nack*) и восстановления зависших задач; выявление и устранение логических ошибок в алгоритмах маршрутизации сообщений и механизмах валидации токенов; оценка консистентности данных между *PostgreSQL* и *Redis* при параллельном извлечении задач несколькими потребителями; проверка корректности обработки граничных состояний – истечения таймаута видимости, исчерпания попыток обработки и внезапной остановки воркера. Каждая из перечисленных задач соответствует отдельному классу потенциальных отказов, выявленных в ходе проектирования архитектуры системы.

Для проверки системы применялся итерационный подход на базе фреймворка *PHPUnit* [12]. Разработан комплекс *Feature*-тестов, покрывающий все аспекты жизненного цикла сообщений и проверяющий строгую консистентность между слоями хранения данных. В отличие от модульных тестов, проверяющих компоненты в изоляции, *Feature*-тесты выполняются против реального окружения с поднятыми экземплярами *PostgreSQL* и *Redis*, что позволяет выявлять ошибки, возникающие исключительно на границах взаимодействия между подсистемами. Каждый тест выполняется в рамках изолированной транзакции с откатом после завершения, что гарантирует независимость сценариев друг от друга и воспроизводимость результатов при любом порядке запуска набора.

Базовый сценарий успешной обработки подтверждён тестом *test_consume_ack_marks_message_done*: воркер потребляет задачу с таймаутом видимости 30 секунд и возвращает валидный *ack_token* вместе с *JSON*-результатом работы – *API* возвращает *HTTP 200*, статус сообщения сменяется на *DONE*, поле *ack_token_hash* обнуляется, а запись удаляется из множества *processing* в *Redis*. Этот тест является фундаментальным: он подтверждает корректность основного пути обработки, на котором строится вся остальная логика системы.

Защита от перезаписи при истечении токена проверена тестом `test_expired_ack_token_is_rejected`: токен принудительно удаляется из Redis, после чего попытка подтверждения отклоняется с кодом HTTP 409 Conflict, а защитный механизм предотвращает любые изменения в базе данных. Данный сценарий моделирует реальную ситуацию, при которой воркер зависает на длительное время и пытается подтвердить обработку уже после того, как система считает задачу потерянной и вернула её в очередь – без подобной защиты это привело бы к двойной обработке сообщения.

Поведение системы при неудачной обработке с оставшимися попытками проверено тестом `test_nack_requeues_message_when_attempts_remain`: задача с `max_attempts=2` завершается ошибкой при установленном флаге `requeue=true`, счётчик `attempts` инкрементируется, после чего задача немедленно возвращается в *Redis ready queue* для повторной выдачи следующему доступному потребителю. Истощение попыток покрыто тестом `test_message_moves_to_dead_after_max_attempts`: статус сменяется на *DEAD*, задача навсегда удаляется из всех структур маршрутизации *Redis*, а связанные токены уничтожаются, исключая возможность любых последующих операций над сообщением. Устойчивость к внезапной остановке воркера подтверждена тестом `test_expired_processing_message_is_recovered`: служба *MessageRecoveryService* обнаруживает зависшую задачу в *Redis ZSET* по истёкшему полю `reserved_until` и возвращает её в статус *PENDING*, помещая обратно в очередь готовности без потери данных и без вмешательства оператора.

Отдельного внимания заслуживает проверка поведения системы при одновременном обращении нескольких потребителей к одной очереди. Атомарность операции извлечения обеспечивается на уровне *Redis* с использованием транзакций, что исключает выдачу одного и того же сообщения двум воркерам одновременно. Тесты, моделирующие конкурентный доступ, подтвердили отсутствие состояний гонки: каждое сообщение в любой момент времени принадлежит ровно одному потребителю, а его статус в *PostgreSQL* и позиция в структурах *Redis* остаются строго согласованными.

Проведённый комплекс испытаний подтвердил надёжность и полную функциональную готовность системы к использованию. Все тесты пройдены успешно, полное отсутствие состояний гонки при параллельном извлечении задач подтверждено, а стопроцентная консистентность данных обеспечена: при любых ошибках на стороне клиента ядро системы блокирует транзакции в *PostgreSQL* и генерирует соответствующие HTTP-исключения уровня 409 Conflict. На основании полученных результатов программный комплекс может быть рекомендован к эксплуатации в системах, требующих надёжной фоновой обработки задач с гарантией доставки.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Настоящее руководство описывает программные интерфейсы взаимодействия для интеграции сторонних микросервисов с разработанным брокером сообщений, а также порядок работы с графической панелью управления. Базовый префикс всех маршрутов *API*: */api*. Формат обмена данными во всех запросах и ответах – *application/json*.

5.1 Спецификация REST API

Интерфейс управления очередями предоставляет четыре операции. Создание новой логической очереди выполняется методом *POST /queues* с передачей уникального имени очереди в теле запроса. Получение списка всех существующих очередей вместе с их агрегированной статистикой по статусам осуществляется методом *GET /queues*. Изменение статуса очереди – приостановка или возобновление обработки – выполняется методом *PATCH /queues/{name}* с передачей целевого статуса. Удаление очереди выполняется методом *DELETE /queues/{name}*; если очередь содержит необработанные задачи, обязательна передача параметра *force=1*, без которого система отклонит запрос в целях защиты данных.

Жизненный цикл сообщений реализован через четыре эндпоинта. Публикация сообщения выполняется методом *POST /queues/{name}/messages*; тело запроса содержит полезную нагрузку в произвольном формате *JSON*, уровень приоритета, задержку перед доставкой в секундах и максимальное число попыток обработки. Потребление задачи выполняется методом *POST /queues/{name}/messages/consume* с указанием идентификатора воркера и желаемого таймаута видимости в секундах; в ответ система возвращает объект задачи и уникальный токен подтверждения. Подтверждение успешной обработки выполняется методом *POST /messages/{id}/ack* с передачей токена и результата работы в формате *JSON*. Сообщение об ошибке обработки отправляется методом *POST /messages/{id}/nack* с указанием токена, текста ошибки и флага повторной постановки в очередь.

Таблица 5.1 – Спецификация *REST API*

Метод / URI	Описание и тело запроса
<i>POST /queues</i>	Создание очереди. <i>Payload</i> : {"name": "mail_queue"}
<i>GET /queues</i>	Список очередей с агрегированной статистикой
<i>PATCH /queues/{name}</i>	Смена статуса. <i>Payload</i> : {"status": "paused"} или {"status": "active"}

Метод / URI	Описание и тело запроса
DELETE /queues/{name}	Удаление. Параметр <i>?force=1</i> при наличии задач обязателен
POST /queues/{name}/messages	Публикация. <i>Payload: payload, priority, delay seconds, max_attempts</i>
POST /queues/{name}/messages/consume	Потребление. <i>Payload: consumer_id, visibility_timeout</i> . Возвращает задачу и <i>ack token</i>
POST /messages/{id}/ack	Подтверждение. <i>Payload: ack_token, result</i>
POST /messages/{id}/nack	Отказ. <i>Payload: ack_token, error, requeue</i>

5.2 Описание графического интерфейса панели управления

Панель управления представляет собой одностраничное веб-приложение, доступное администратору после прохождения аутентификации на основе сессий *Laravel Sanctum*. Интерфейс организован как единый экран, все элементы которого обновляются в режиме реального времени посредством *WebSocket*-соединения без перезагрузки страницы.

Верхняя часть экрана содержит сводную панель ключевых метрик брокера. Четыре виджета отображают общее количество сообщений в системе, число задач в состоянии ожидания, число задач в процессе обработки и число задач, завершившихся ошибкой. Значения виджетов обновляются мгновенно при каждом изменении состояния системы.

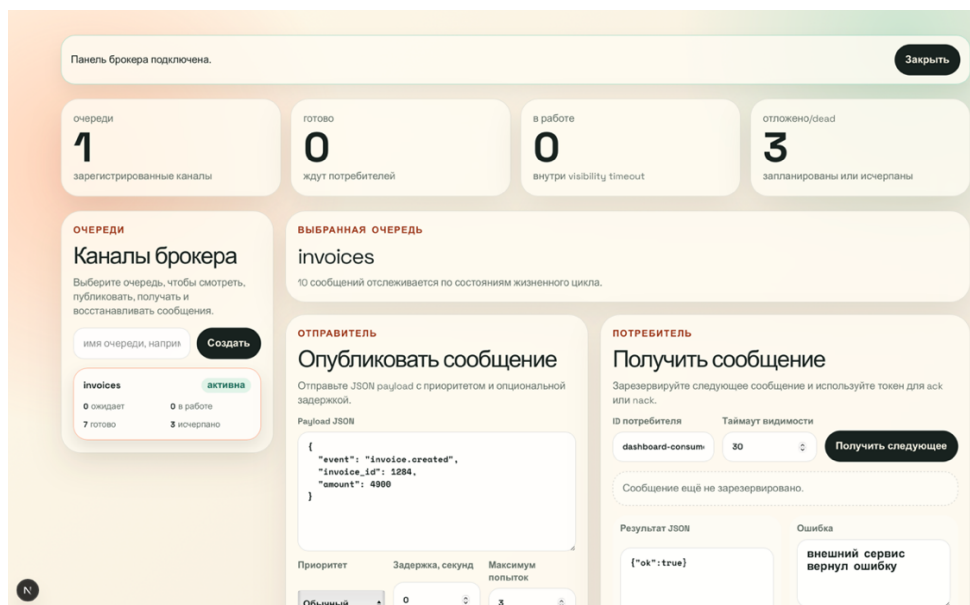


Рисунок 5.1 – Главный экран панели управления: сводные метрики брокера

Центральную часть интерфейса занимает таблица очередей. Каждая строка таблицы соответствует одной логической очереди и отображает её имя, текущий статус, счётчики сообщений по каждому из статусов и элемент управления для переключения состояния. Администратор может приостановить любую очередь кликом на переключатель статуса непосредственно в строке таблицы, не покидая текущего экрана.

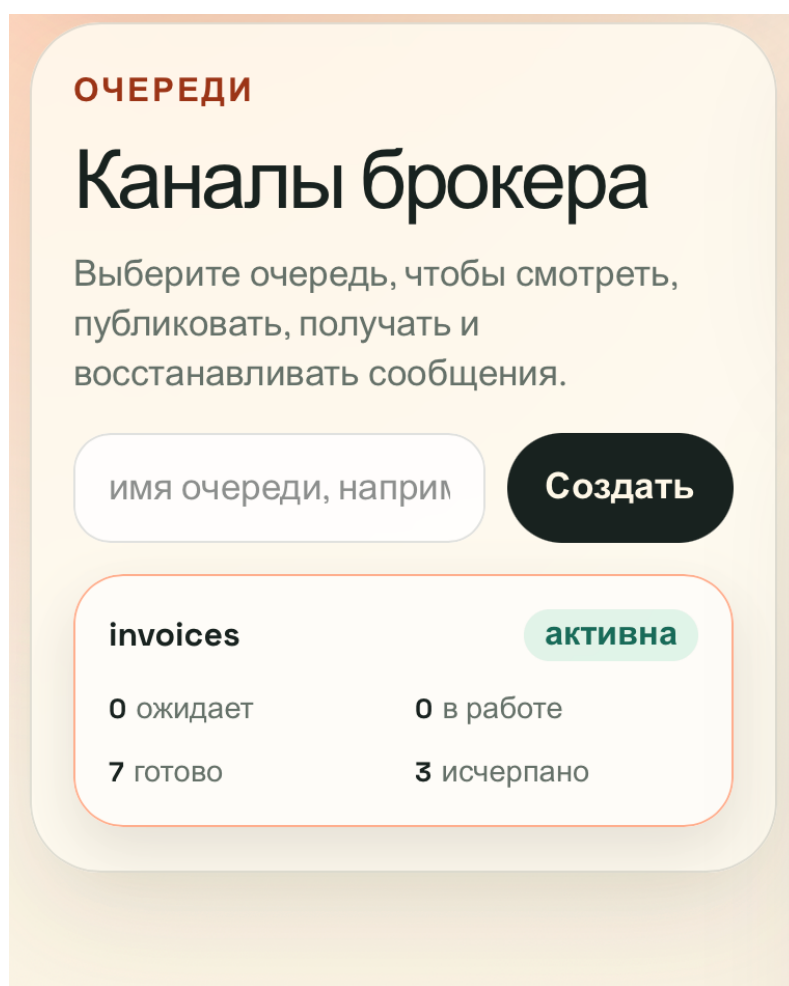


Рисунок 5.2 – Таблица очередей с переключателями статуса

В нижней части экрана расположен журнал последних сообщений. Таблица отображает идентификатор, имя очереди, приоритет, текущий статус и временные метки для каждого из последних поступивших сообщений. Столбец статуса наглядно отражает текущее положение задачи в жизненном цикле: *pending* означает ожидание в очереди, *processing* – активную обработку потребителем, *done* – успешное завершение, *dead* – исчерпание допустимого числа попыток. Записи добавляются в таблицу в режиме реального времени по мере публикации новых

задач, что позволяет наблюдать за динамикой поступления сообщений без необходимости обновления страницы вручную.

СООБЩЕНИЯ

Журнал сообщений

Все статусы ▾ Все приоритет ▾

ID	СТАТУС	ПРИОРИТЕТ	ПОПЫТКИ	ПОТРЕБИТЕЛЬ	ЗАРЕЗЕРВИРОВАНО ДО	PAYLOAD / РЕЗУЛЬТАТ
#21	Выполнено	Обычный	2/3	dashboard-consumer-1	-	» Открыть
#20	Выполнено	Обычный	1/3	dashboard-consumer-1	-	» Открыть
#19	Выполнено	Обычный	2/3	dashboard-consumer-1	-	» Открыть
#18	Выполнено	Обычный	1/3	billing-worker-1	-	» Открыть
#17	Выполнено	Обычный	1/3	dashboard-consumer-1	-	» Открыть
#16	Исчерпано	Обычный	2/5	dashboard-consumer-1	-	» Открыть
#15	Исчерпано	Обычный	3/3	dashboard-consumer-1	-	» Открыть
#14	Исчерпано	Обычный	3/3	dashboard-consumer-1	-	» Открыть
#13	Выполнено	Обычный	3/3	dashboard-consumer-1	-	» Открыть
#12	Выполнено	Обычный	1/3	dashboard-consumer-1	-	» Открыть

Рисунок 5.3 – Журнал последних сообщений в режиме реального времени

Отдельным элементом управления на экране представлена кнопка принудительного запуска службы восстановления. При её нажатии система выполняет обращение к эндпоинту *POST /broker/recover*, инициируя немедленное сканирование зависших и просроченных задач с их последующей повторной активацией. Необходимость данного механизма обусловлена возможными сбоями на стороне потребителей: если задача была получена, но подтверждение так и не поступило до истечения таймаута видимости, она остаётся в промежуточном состоянии и без принудительного восстановления не вернётся в очередь готовности. Служба восстановления выявляет подобные задачи и возвращает их в активную маршрутизацию. После завершения операции панель мгновенно отражает изменившееся распределение задач по статусам.

Перед выполнением основных запросов браузер автоматически отправляет служебный *OPTIONS*-запрос, предназначенный для *CORS*-проверки. Необходимость данной проверки обусловлена тем, что клиентская часть и серверный API функционируют на различных источниках происхождения (*origin*), а основные запросы передают данные в формате *JSON* методом *POST*. Запрос *OPTIONS* не реализует бизнес-логику приложения: он не создаёт очередь, не отправляет и не получает сообщения, а лишь устанавливает факт разрешения сервером последующего запроса.

На сетевом дампе зафиксировано обращение к серверному API на порту 8000 с целью создания новой очереди. После успешного прохождения *CORS*-проверки выполняется основной запрос *POST /api/queues*. В теле запроса передаётся наименование создаваемой очереди. Сервер обрабатывает запрос, формирует запись очереди в базе данных *PostgreSQL*, регистрирует её в *Redis* и возвращает клиенту данные созданной очереди.

```
OPTIONS /api/queues HTTP/1.1
HTTP/1.0 204 No Content
POST /api/queues HTTP/1.1 , JSON (application/json)
HTTP/1.1 201 Created , JSON (application/json)
```

Рисунок 5.6 – Запросы на создание новой очереди

```
JavaScript Object Notation: application/json
▼ Object
  ▼ Member: name
    [Path with value: /name:invoices]
    [Member with value: name:invoices]
    String value: invoices
    Key: name
    [Path: /name]
```

Рисунок 5.7 – Содержание запроса

JavaScript Object Notation: application/json

```
✓ Object
  ✓ Member: data
    ✓ Object
      > Member: id
      > Member: name
      > Member: status
      > Member: stats
      > Member: created_at
      > Member: updated_at
    Key: data
    [Path: /data]
```

Рисунок 5.8 – Содержание ответа

После создания очереди издатель приступает к публикации сообщения. В соответствующем разделе интерфейса указывается полезная нагрузка сообщения в формате *JSON*, а также, при необходимости, уровень приоритета, задержка перед доставкой в секундах и максимально допустимое количество попыток обработки. Уровень приоритета определяет порядок извлечения сообщений из очереди относительно других, задержка позволяет отложить передачу сообщения потребителю на заданный интервал времени, а ограничение на количество попыток обработки исключает бесконечное повторное выполнение задач при систематических сбоях. По завершении заполнения всех параметров сообщение публикуется в очередь.

На сетевом дампе зафиксирован обмен данными между клиентской частью и серверным *API* в процессе публикации сообщения. Запрос направляется к *API*-серверу на порту 8000. После прохождения служебного *OPTIONS*-запроса, подтверждающего допустимость кросс-доменного обращения, отправляется основной запрос *POST /api/queues/{queue}/messages*, в теле которого передаются данные публикуемого сообщения. Получив запрос, сервер создаёт соответствующую запись в базе данных *PostgreSQL* и помещает сообщение в очередь *Redis*, после чего оно становится доступным для потребителей.

ОТПРАВИТЕЛЬ

Опубликовать сообщение

Отправьте JSON payload с приоритетом и опциональной задержкой.

Payload JSON

```
{  
  "event": "invoice.created",  
  "invoice_id": 1284,  
  "amount": 4900  
}
```

Приоритет	Задержка, секунд	Максимум попыток
Обычный	0	3

Опубликовать в invoices

Рисунок 5.9 – Форма публикации сообщения: указание *payload*, приоритета и параметров доставки

```
OPTIONS /api/queues/invoices/messages HTTP/1.1  
HTTP/1.0 204 No Content  
POST /api/queues/invoices/messages HTTP/1.1 , JSON (application/json)  
HTTP/1.1 201 Created , JSON (application/json)
```

Рисунок 5.10 – Запросы на создание сообщения

```
JavaScript Object Notation: application/json
└─ Object
  └─ Member: payload
    └─ Object
      > Member: event
      > Member: invoice_id
      > Member: amount
      Key: payload
      [Path: /payload]
    > Member: priority
    > Member: delay_seconds
    > Member: max_attempts
```

Рисунок 5.11 – Содержание тела запроса для создания сообщения

```
JavaScript Object Notation: application/json
└─ Object
  └─ Member: data
    └─ Object
      > Member: id
      > Member: queue_id
      > Member: queue_name
      > Member: payload
      > Member: status
      > Member: priority
      > Member: attempts
      > Member: max_attempts
      > Member: available_at
      > Member: reserved_until
      > Member: consumer_id
      > Member: result
      > Member: error
      > Member: completed_at
      > Member: created_at
      > Member: updated_at
      Key: data
```

Рисунок 5.12 – Содержание тела ответа

На стороне потребителя для получения задачи необходимо указать идентификатор воркера и желаемый таймаут видимости – период времени, в течение которого задача считается заблокированной за данным потребителем. После отправки запроса система атомарно извлекает из очереди первое доступное сообщение в соответствии с его приоритетом и возвращает потребителю объект задачи, содержащий полезную нагрузку и уникальный токен подтверждения.

ПОТРЕБИТЕЛЬ

Получить сообщение

Зарезервируйте следующее сообщение и используйте токен для ask или nack.

ID потребителя

dashboard-consum

Таймаут видимости

30

Получить следующее

ЗАРЕЗЕРВИРОВАНО СООБЩЕНИЕ #23

```
{
  "event": "invoice.created",
  "invoice_id": 1284,
  "amount": 4900
}
```

Ask токен

weekKDLZIOEd1azSuffNaxuP70lu8tfdoZPMJ7ThEfqVnLj04c

Результат JSON

```
{"ok": true}
```

Подтвердить успех

Ошибка

внешний сервис
вернул ошибку

☒ Вернуть в очередь для новой попытки

Nack / ошибка

Рисунок 5.13 – Результат операции потребления: объект сообщения и *ack_token*

На сетевом дампе зафиксирован обмен данными с серверным *API* при получении сообщения из очереди. Запрос направляется к *API*-серверу на порту 8000. После служебного *OPTIONS*-запроса выполняется основной запрос *POST /api/queues/{queue}/messages/consume*. Сервер резервирует сообщение за потребителем, переводит его в состояние *processing* и возвращает данные сообщения вместе с *ack_token*, предназначенным для подтверждения факта обработки.

```
OPTIONS /api/queues/another/messages/consume HTTP/1.1
HTTP/1.0 204 No Content
POST /api/queues/another/messages/consume HTTP/1.1 , JSON (application/json)
HTTP/1.1 200 OK , JSON (application/json)
```

Рисунок 5.14 – Запросы на получение сообщения

```
JavaScript Object Notation: application/json
  ✓ Object
    > Member: consumer_id
    ✓ Member: visibility_timeout
```

Рисунок 5.15 – Содержание тела запроса на получение сообщения

```
JavaScript Object Notation: application/json
  ✓ Object
    ✓ Member: data
      ✓ Object
        > Member: message
        > Member: ack_token
        > Member: empty
      Key: data
      [Path: /data]
```

Рисунок 5.16 – Содержание тела ответа на получение сообщения

```
OPTIONS /api/messages/29/ack HTTP/1.1
HTTP/1.0 204 No Content
POST /api/messages/29/ack HTTP/1.1 , JSON (application/json)
HTTP/1.1 200 OK , JSON (application/json)
```

Рисунок 5.17 – Запрос на подтверждение сообщения *ack*

```

JavaScript Object Notation: application/json
  Object
    Member: ack_token
      [Path with value: /ack_token:YZrR9ZQ0y5MWS0V2VuxDQ94ZXdrGCd84ljwhXaexGLqUdZPL]
      [Member with value: ack_token:YZrR9ZQ0y5MWS0V2VuxDQ94ZXdrGCd84ljwhXaexGLqUdZPL]
      String value: YZrR9ZQ0y5MWS0V2VuxDQ94ZXdrGCd84ljwhXaexGLqUdZPL
      Key: ack_token
      [Path: /ack_token]
    Member: result
      Object
        Member: ok
        Key: result
        [Path: /result]

```

Рисунок 5.18 – Содержание тела запроса на подтверждение

```

OPTIONS /api/messages/30/nack HTTP/1.1
HTTP/1.0 204 No Content
POST /api/messages/30/nack HTTP/1.1 , JSON (application/json)
HTTP/1.1 200 OK , JSON (application/json)

```

Рисунок 5.19 – Запросы на отмену сообщения *nack*

```

JavaScript Object Notation: application/json
  Object
    Member: ack_token
      [Path with value: /ack_token:5iFMiTjBJnBtXVRZqXUPSoRALpCWWq56HFz0nVKSTf0cEJ9v]
      [Member with value: ack_token:5iFMiTjBJnBtXVRZqXUPSoRALpCWWq56HFz0nVKSTf0cEJ9v]
      String value: 5iFMiTjBJnBtXVRZqXUPSoRALpCWWq56HFz0nVKSTf0cEJ9v
      Key: ack_token
      [Path: /ack_token]
    Member: error
    Member: queue

```

Рисунок 5.20 – Содержание тела запроса на отмену

Получив задачу, потребитель обязан в течение таймаута видимости сообщить системе о результате её обработки. Данное ограничение по времени является ключевым механизмом защиты от потери сообщений: если потребитель не направит ответ до истечения таймаута, система автоматически снимает резервирование и возвращает сообщение в очередь готовности для повторного получения другим потребителем.

При успешном выполнении задачи потребитель отправляет подтверждение, содержащее токен и результирующие данные обработки, после чего задача переводится в финальный статус *done* и исключается из активной маршрутиза-

ции. При возникновении ошибки потребитель направляет отказ с указанием причины и флага повторной постановки в очередь. Если флаг установлен, задача возвращается в очередь готовности для повторной попытки обработки с учётом заданного лимита попыток; если флаг не установлен либо лимит попыток исчерпан, задача переводится в статус *dead* и исключается из дальнейшей маршрутизации. Сообщения в статусе *dead* сохраняются в журнале и доступны для последующего анализа причин сбоев.

По завершении операции запись о сообщении отображается в журнале с полной информацией: идентификатором, именем очереди, приоритетом, итоговым статусом, числом использованных попыток и временными метками всех ключевых событий жизненного цикла.

Журнал сообщений						
		Все статусы		Все приоритет		
ID	СТАТУС	ПРИОРИТЕТ	ПОПЫТКИ	ПОТРЕБИТЕЛЬ	ЗАРЕЗЕРВИРОВАНО ДО	PAYLOAD / РЕЗУЛЬТАТ
#24	Выполнено	Обычный	1/3	dashboard-consumer-1	-	» Открыть

Рисунок 5.21 – Отображение обработанного сообщения в журнале со всеми метаданными

Таким образом, представленный сценарий наглядно иллюстрирует прозрачность и надёжность механизмов обработки асинхронных задач. Использование токенов подтверждения и таймаутов видимости защищает систему от потери данных при внезапных сбоях на стороне узлов-обработчиков, реализуя строгую семантику доставки «как минимум один раз» (*at-least-once*). Разделение итоговых статусов на *done* и *dead* позволяет чётко разграничивать успешно завершённые задачи и задачи, требующие вмешательства оператора, что упрощает диагностику и сопровождение системы в производственной среде.

В свою очередь, накопленные метаданные в журнале событий служат фундаментом для построения систем мониторинга: они позволяют выявлять узкие места в производительности, настраивать автоматическое оповещение при аномальном росте числа сообщений в статусе *dead* и принимать обоснованные решения о необходимости горизонтального масштабирования пула потребителей в условиях пиковых нагрузок. Это обеспечивает высокую отказоустойчивость и стабильность работы всего программного комплекса.

ЗАКЛЮЧЕНИЕ

В процессе выполнения курсового проекта была спроектирована, реализована и покрыта автоматизированными тестами масштабируемая программная система управления очередями сообщений. Разработанный брокер представляет собой полноценный производственный инструмент, сочетающий высокую производительность, гарантии доставки и удобство интеграции в существующие микросервисные комплексы.

Ключевым инженерным результатом проекта является разработка гибридной транзакционной архитектуры хранения данных. Чёткое разделение зон ответственности между реляционной базой данных *PostgreSQL*, выступающей единственным авторитетным источником истины и гарантирующей долговременную сохранность данных, и высокопроизводительным хранилищем *Redis*, реализующим сверхбыстрый маршрутизатор задач в оперативной памяти, позволило достичь времени обработки операций публикации и потребления на уровне единиц миллисекунд без каких-либо компромиссов в части консистентности и надёжности.

Проблема уникальности выдачи задач при высоком уровне параллелизма была решена посредством атомарных *Lua*-скриптов, исполняемых непосредственно внутри движка *Redis*. Данный подход полностью исключает состояния гонки между конкурирующими воркерами без введения распределённых блокировок и без дополнительной нагрузки на реляционную базу данных. Безопасность операций подтверждения на стороне клиентских воркеров обеспечивается системой криптографических токенов, хранимых в хэшированном виде, что защищает систему от несанкционированного управления чужими задачами даже в случае компрометации базы данных.

Внедрение протокола *WebSocket* на базе сервера *Laravel Verbb* и клиентской библиотеки *Laravel Echo* позволило создать отзывчивый административный интерфейс, мгновенно отражающий изменения состояния брокера без избыточных затрат на периодический опрос сервера. Панель мониторинга обеспечивает оперативный контроль над всеми очередями, распределением нагрузки и жизненным циклом отдельных сообщений в режиме реального времени.

Все цели и задачи, поставленные в начале курсовой работы, выполнены в полном объёме. Разработанный *REST API* соответствует современным стандартам микросервисной интеграции, а реализованные механизмы гарантированной доставки, повторных попыток и автоматического восстановления зависших задач обеспечивают уровень надёжности, достаточный для промышленной эксплуатации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Крепс, Дж. Журнал: Что должен знать каждый инженер-программист о единой абстракции данных в реальном времени [Электронный ресурс] / Дж. Крепс. – Режим доступа: <https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>. – Дата доступа: 10.05.2026.
- [2] Ричардсон, К. Паттерны микросервисов / К. Ричардсон. – Шелтер-Айленд : *Manning Publications*, 2018. – 520 с.
- [3] Наркхеде, Н. *Kafka*: Исчерпывающее руководство: Обработка данных в реальном времени и потоки в масштабе / Н. Наркхеде, Г. Шапира, Т. Палино. – Себастопол : *O'Reilly Media*, 2017. – 320 с.
- [4] Зайцев, П. Сравнение платформ обмена сообщениями для больших данных: Оценка *Apache Kafka*, *RabbitMQ* и *Redis* [Электронный ресурс] / П. Зайцев // *ResearchGate*. – 2024. – Режим доступа: <https://www.researchgate.net/publication/398353443>. – Дата доступа: 12.05.2026.
- [5] Филдинг, Р. Т. Архитектурные стили и проектирование сетевых программных архитектур : дис. ... д-ра филос. / Р. Т. Филдинг. – Ирвин : Калифорнийский университет, 2000. – 162 с.
- [6] Документация *Laravel 11.x*: Трансляция (*Broadcasting*) [Электронный ресурс]. – Режим доступа: <https://laravel.com/docs/11.x/broadcasting>. – Дата доступа: 14.05.2026.
- [7] Документация *Redis*: Типы данных [Электронный ресурс]. – Режим доступа: <https://redis.io/docs/latest/develop/data-types/>. – Дата доступа: 14.05.2026.
- [8] Документация *Postgres Pro*: Типы *JSON* [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgrespro/current/datatype-json>. – Дата доступа: 15.05.2026.
- [9] Фетте, И. *RFC 6455*: Протокол *WebSocket* [Электронный ресурс] / И. Фетте, А. Мельников. – *IETF*, 2011. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc6455>. – Дата доступа: 15.05.2026.
- [10] Документация *Redis*: Скрипты на *Lua* [Электронный ресурс]. – Режим доступа: <https://redis.io/docs/latest/develop/programmability/eval-intro/>. – Дата доступа: 16.05.2026.
- [11] Документация *Laravel 11.x*: Очереди (*Queues*) [Электронный ресурс]. – Режим доступа: <https://laravel.com/docs/11.x/queues>. – Дата доступа: 16.05.2026.
- [12] Документация *PHPUnit* [Электронный ресурс]. – Режим доступа: <https://phpunit.de/documentation.html>. – Дата доступа: 17.05.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода с комментариями

Проект реализует простой *broker* очередей сообщений:

1. *Queue* – очередь, в которую публикуются сообщения.
2. *Message* – сообщение внутри очереди.
3. *Redis* – быстрая структура для выбора следующего сообщения, хранения *delayed/processing*-состояний и *ack*-токенов.
4. База данных – постоянное хранилище очередей и сообщений.
5. *API controllers* – *HTTP*-интерфейс для *frontend*/клиентов.
6. *Services* – бизнес-логика брокера.
7. *Resources* – форматирование *JSON*-ответов.
8. *Requests* – валидация входящих данных.

1. Статусы сообщений

Файл: ``app/Enum/MessageStatuses.php``

```
```php
<?php

namespace App\Enum;

/**
 * Enum MessageStatuses описывает все возможные состояния сообщения.
 *
 * Enum удобнее обычных строк, потому что:
 * - нельзя случайно использовать несуществующий статус;
 * - Laravel умеет автоматически приводить строку из БД к enum;
 * - код становится понятнее: MessageStatuses::PENDING вместо 'pending'.
 */
enum MessageStatuses: string
{
 /**
 * Сообщение готово к обработке и лежит в ready-очереди Redis.
 */
 case PENDING = 'pending';

 /**
 * Сообщение отложено и станет доступно позже.
 * Такие сообщения хранятся в Redis sorted set delayed.
 */
 case DELAYED = 'delayed';

 /**
 * Сообщение уже выдано consumer-у и ждёт ack/nack.
 */
 case PROCESSING = 'processing';

 /**
 * Сообщение успешно обработано.
 */
 case DONE = 'done';

 /**
 * Consumer сообщил об ошибке, но сообщение ещё можно вернуть в очередь.
 */
 case FAILED = 'failed';

 /**
 * Сообщение больше не будет обрабатываться.
 * Обычно сюда попадают сообщения после исчерпания max_attempts.
 */
 case DEAD = 'dead';
}
```
```

2. Статусы очередей

Файл: `app/Enum/QueueStatuses.php`

```
```php
<?php

namespace App\Enum;

/**
 * Enum QueueStatuses описывает состояние очереди.
 */
enum QueueStatuses: string
{
 /**
 * Очередь активна: из неё можно читать сообщения.
 */
 case ACTIVE = 'active';

 /**
 * Очередь поставлена на паузу: consume запрещён.
 */
 case PAUSED = 'paused';
}
```
```

3. Модель Queue

Файл: `app/Models/Queue.php`

```
```php
<?php

namespace App\Models;

use App\Enum\QueueStatuses;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

/**
 * Queue - Eloquent-модель таблицы queues.
 *
 * Одна очередь содержит много сообщений.
 */
class Queue extends Model
{
 use HasFactory;

 /**
 * Поля, которые разрешено массово заполнять через create/update.
 */
 protected $fillable = [
 'name',
 'status',
];

 /**
 * Laravel автоматически преобразует поле status:
 * - из строки ВД в QueueStatuses;
 * - из QueueStatuses обратно в строку при сохранении.
 */
 protected function casts(): array
 {
 return [
 'status' => QueueStatuses::class,
];
 }
}

/**
```

```

 * Связь: одна очередь имеет много сообщений.
 *
 * Использование:
 * $queue->messages()->create([...])
 * $queue->messages()->where('status', 'pending')->get()
 */
 public function messages(): HasMany
 {
 return $this->hasMany(Message::class);
 }
}
...

```

---

## ## 4. Модель Message

Файл: `app/Models/Message.php`

```

...php
<?php

namespace App\Models;

use App\Enum\MessageStatuses;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

/**
 * Message - Eloquent-модель таблицы messages.
 *
 * Сообщение принадлежит одной очереди и содержит:
 * - payload: полезную нагрузку;
 * - priority: приоритет;
 * - attempts/max_attempts: счётчик попыток;
 * - ack_token_hash: hash токена подтверждения;
 * - reserved_until: срок, до которого consumer должен подтвердить обработку.
 */
class Message extends Model
{
 use HasFactory;

 protected $fillable = [
 'queue_id',
 'payload',
 'priority',
 'status',
 'attempts',
 'max_attempts',
 'available_at',
 'reserved_until',
 'consumer_id',
 'ack_token_hash',
 'result',
 'error',
 'completed_at',
];

 /**
 * Преобразования типов:
 * - payload/result хранятся как JSON, в PHP становятся массивами;
 * - status становится MessageStatuses enum;
 * - даты становятся Carbon-объектами.
 */
 protected function casts(): array
 {
 return [
 'payload' => 'array',
 'result' => 'array',
 'status' => MessageStatuses::class,
 'available_at' => 'datetime',
 'reserved_until' => 'datetime',
 'completed_at' => 'datetime',
];
 }
}

```

```

];
}

/**
 * Связь: сообщение принадлежит одной очереди.
 */
public function queue(): BelongsTo
{
 return $this->belongsTo(Queue::class);
}
}
...

```

---

## ## 5. RedisBroker - низкоуровневая работа с Redis

Файл: `app/Services/Broker/RedisBroker.php`

```

` `` php
<?php

namespace App\Services\Broker;

use App\Models\Queue;
use Illuminate\Support\Facades\Redis;

/**
 * RedisBroker инкапсулирует все Redis-операции.
 *
 * Важно: бизнес-логика находится не здесь, а в сервисах.
 * Этот класс только знает, какие Redis-ключи использовать и какие команды вызывать.
 */
class RedisBroker
{
 /**
 * Порядок приоритетов важен.
 * consumeNext сначала ищет high, потом normal, потом low.
 */
 public const PRIORITIES = ['high', 'normal', 'low'];

 /**
 * Регистрирует очередь в общем Redis-set.
 * Это вспомогательный список существующих очередей.
 */
 public function registerQueue(Queue $queue): void
 {
 Redis::sadd($this->queuesKey(), $queue->name);
 }

 /**
 * Удаляет очередь из Redis и очищает все связанные структуры:
 * - delayed sorted set;
 * - processing sorted set;
 * - ready lists по каждому приоритету.
 */
 public function unregisterQueue(string $queueName): void
 {
 Redis::srem($this->queuesKey(), $queueName);
 Redis::del($this->delayedKey($queueName));
 Redis::del($this->processingKey($queueName));

 foreach (self::PRIORITIES as $priority) {
 Redis::del($this->readyKey($queueName, $priority));
 }
 }

 /**
 * Кладёт сообщение в ready-очередь.
 *
 * Redis list используется как очередь:
 * - LPUSH добавляет слева;
 * - RPOP забирает справа.
 */

```

```

public function enqueueReady(string $queueName, int $messageId, string $priority = 'normal'):
void
{
 Redis::lpush($this->readyKey($queueName, $priority), (string) $messageId);
}

/**
 * Кладёт отложенное сообщение в sorted set.
 *
 * Score = timestamp, когда сообщение станет доступно.
 * Потом recovery выбирает все сообщения, у которых score <= now.
 */
public function enqueueDelayed(string $queueName, int $messageId, int $availableAt): void
{
 Redis::zadd($this->delayedKey($queueName), $availableAt, (string) $messageId);
}

/**
 * Атомарно забирает следующее сообщение из ready-очереди.
 *
 * Метод использует Lua-скрипт, чтобы операция была атомарной:
 * 1. пройти по ready-ключам в порядке high -> normal -> low;
 * 2. взять первое найденное сообщение через RPOP;
 * 3. сразу добавить его в processing sorted set с deadline;
 * 4. вернуть id сообщения.
 *
 * Если делать это несколькими отдельными Redis-командами,
 * два consumer-а могли бы получить одно и то же сообщение.
 */
public function consumeNext(string $queueName, int $deadline): ?int
{
 $readyKeys = array_map(
 fn (string $priority): string => $this->readyKey($queueName, $priority),
 self::PRIORITIES,
);

 $keys = [...$readyKeys, $this->processingKey($queueName)];

 $script = <<<'LUA'
for index = 1, #KEYS - 1 do
local message_id = redis.call('RPOP', KEYS[index])
if message_id then
redis.call('ZADD', KEYS[#KEYS], ARGV[1], message_id)
return message_id
end
end
return nil
LUA;

 $messageId = Redis::command('eval', [$script, count($keys), ...$keys, (string) $deadline]);

 return $messageId === null ? null : (int) $messageId;
}

/**
 * Добавляет сообщение в processing вручную.
 * В текущей логике consumeNext уже делает это атомарно.
 */
public function markProcessing(string $queueName, int $messageId, int $deadline): void
{
 Redis::zadd($this->processingKey($queueName), $deadline, (string) $messageId);
}

/**
 * Убирает сообщение из processing.
 * Вызывается после ack/nack или при recovery.
 */
public function removeProcessing(string $queueName, int $messageId): void
{
 Redis::zrem($this->processingKey($queueName), (string) $messageId);
}

/**

```

```

 * Убирает сообщение из delayed.
 * Вызывается, когда delayed-сообщение стало ready.
 */
 public function removeDelayed(string $queueName, int $messageId): void
 {
 Redis::zrem($this->delayedKey($queueName), (string) $messageId);
 }

 /**
 * Возвращает id delayed-сообщений, которые уже можно обработать.
 */
 public function dueDelayed(string $queueName, int $now, int $limit = 100): array
 {
 return array_map('intval', Redis::zrangebyscore($this->delayedKey($queueName), '-inf', $now,
['limit' => [0, $limit]]));
 }

 /**
 * Возвращает id processing-сообщений, у которых истёк visibility timeout.
 */
 public function expiredProcessing(string $queueName, int $now, int $limit = 100): array
 {
 return array_map('intval', Redis::zrangebyscore($this->processingKey($queueName), '-inf',
$now, ['limit' => [0, $limit]]));
 }

 /**
 * Сохраняет ack-token во временный Redis-ключ.
 *
 * TTL нужен, чтобы токен автоматически исчез,
 * если consumer не подтвердил сообщение вовремя.
 */
 public function setAckToken(int $messageId, string $token, int $ttl): void
 {
 Redis::setex($this->tokenKey($messageId), $ttl, $token);
 }

 /**
 * Получает ack-token из Redis для проверки ack/nack.
 */
 public function getAckToken(int $messageId): ?string
 {
 return Redis::get($this->tokenKey($messageId));
 }

 /**
 * Удаляет ack-token после завершения обработки.
 */
 public function deleteAckToken(int $messageId): void
 {
 Redis::del($this->tokenKey($messageId));
 }

 /**
 * Считает все ready-сообщения по всем приоритетам.
 */
 public function readyCount(string $queueName): int
 {
 return array_sum(array_map(
 fn (string $priority): int => (int) Redis::llen($this->readyKey($queueName, $priority)),
 self::PRIORITIES,
));
 }

 /**
 * Считает delayed-сообщения в Redis.
 */
 public function delayedCount(string $queueName): int
 {
 return (int) Redis::zcard($this->delayedKey($queueName));
 }

 /**
 * Считает processing-сообщения в Redis.

```

```

 */
 public function processingCount(string $queueName): int
 {
 return (int) Redis::zcard($this->processingKey($queueName));
 }

 private function queuesKey(): string
 {
 return 'broker:queues';
 }

 private function readyKey(string $queueName, string $priority): string
 {
 return "broker:queue:{$queueName}:ready:{$priority}";
 }

 private function delayedKey(string $queueName): string
 {
 return "broker:queue:{$queueName}:delayed";
 }

 private function processingKey(string $queueName): string
 {
 return "broker:queue:{$queueName}:processing";
 }

 private function tokenKey(int $messageId): string
 {
 return "broker:message:{$messageId}:token";
 }
}
...

```

---

## ## 6. QueueService - бизнес-логика очередей

Файл: `app/Services/Broker/QueueService.php`

```

```php
<?php

namespace App\Services\Broker;

use App\Enum\MessageStatuses;
use App\Enum\QueueStatuses;
use App\Events\BrokerStateUpdated;
use App\Models\Message;
use App\Models\Queue;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Support\Facades\DB;
use InvalidArgumentException;

/**
 * QueueService управляет очередями:
 * - список;
 * - создание;
 * - обновление статуса;
 * - удаление;
 * - статистика.
 */
class QueueService
{
    public function __construct(
        private readonly RedisBroker $redisBroker,
        private readonly MessageRecoveryService $recoveryService,
    ) {}

    /**
     * Возвращает список очередей.
     *
     * Перед возвратом запускается recovery для каждой очереди,
     * чтобы delayed/expired сообщения были приведены в актуальное состояние.
     */
}

```



```

public function list(): Collection
{
    $queues = Queue::query()->orderBy('name')->get();
    $queues->each(fn (Queue $queue) => $this->recoveryService->recoverQueue($queue));

    return $queues;
}

/**
 * Создает очередь в БД и регистрирует её в Redis.
 *
 * Транзакция гарантирует, что при ошибке БД изменения откатятся.
 */
public function create(array $data): Queue
{
    return DB::transaction(function () use ($data): Queue {
        $queue = Queue::query()->create([
            'name' => $data['name'],
            'status' => QueueStatuses::ACTIVE,
        ]);

        $this->redisBroker->registerQueue($queue);
        event(new BrokerStateUpdated($queue->name));

        return $queue;
    });
}

/**
 * Обновляет статус очереди.
 *
 * Метод поддерживает два варианта входа:
 * - status: active/paused;
 * - paused: true/false.
 */
public function update(Queue $queue, array $data): Queue
{
    if (array_key_exists('paused', $data)) {
        $data['status'] = $data['paused'] ? QueueStatuses::PAUSED->value : QueueStatuses::AC-
TIVE->value;
    }

    if (isset($data['status'])) {
        $queue->status = QueueStatuses::from($data['status']);
        $queue->save();
        event(new BrokerStateUpdated($queue->name));
    }

    return $queue->refresh();
}

/**
 * Удаляет очередь.
 *
 * Если force=false и в очереди есть сообщения - удаление запрещено.
 * Это защита от случайной потери данных.
 */
public function delete(Queue $queue, bool $force = false): void
{
    if (! $force && $queue->messages()->exists()) {
        throw new InvalidArgumentException('Queue is not empty. Pass force=true to delete it.');
```

```

public function stats(?Queue $queue = null): array
{
    if ($queue) {
        $this->recoveryService->recoverQueue($queue);

        return $this->statsForQueue($queue);
    }

    $this->recoveryService->recoverAll();

    return [
        'queues' => Queue::query()->count(),
        'total' => Message::query()->count(),
        'pending' => Message::query()->where('status', MessageStatuses::PENDING)->count(),
        'delayed' => Message::query()->where('status', MessageStatuses::DELAYED)->count(),
        'processing' => Message::query()->where('status', MessageStatuses::PROCESSING)->count(),
        'done' => Message::query()->where('status', MessageStatuses::DONE)->count(),
        'failed' => Message::query()->where('status', MessageStatuses::FAILED)->count(),
        'dead' => Message::query()->where('status', MessageStatuses::DEAD)->count(),
    ];
}

/**
 * Возвращает подробную статистику одной очереди:
 * - количество сообщений в Redis;
 * - количество сообщений в БД по статусам.
 */
public function statsForQueue(Queue $queue): array
{
    $statusCounts = $queue->messages()
        ->select('status', DB::raw('count(*) as total'))
        ->groupBy('status')
        ->pluck('total', 'status')
        ->all();

    return [
        'queue' => $queue,
        'redis' => [
            'ready' => $this->redisBroker->readyCount($queue->name),
            'delayed' => $this->redisBroker->delayedCount($queue->name),
            'processing' => $this->redisBroker->processingCount($queue->name),
        ],
        'messages' => collect(MessageStatuses::cases())
            ->mapWithKeys(fn (MessageStatuses $status): array => [$status->value => (int) ($statusCounts[$status->value] ?? 0)])
            ->all(),
    ];
}
}
...

```

7. MessagePublishService - публикация сообщений

Файл: `app/Services/Broker/MessagePublishService.php`

```

` ``php
<?php

namespace App\Services\Broker;

use App\Enum\MessageStatuses;
use App\Events\BrokerStateUpdated;
use App\Models\Message;
use App\Models\Queue;
use Illuminate\Support\Facades\DB;

/**
 * MessagePublishService отвечает за добавление новых сообщений в очередь.
 */
class MessagePublishService
{
    public function __construct(

```

```

        private readonly RedisBroker $redisBroker,
        private readonly MessageRecoveryService $recoveryService,
    ) {}

    /**
     * Публикует сообщение.
     *
     * Алгоритм:
     * 1. Сначала запускается recovery, чтобы очередь была актуальной.
     * 2. Определяется delay_seconds.
     * 3. Создаётся запись в БД.
     * 4. Если delay > 0 - сообщение кладётся в Redis delayed.
     * 5. Если delay = 0 - сообщение кладётся в Redis ready.
     * 6. Отправляется событие обновления состояния брокера.
     */
    public function publish(Queue $queue, array $data): Message
    {
        $this->recoveryService->recoverQueue($queue);

        return DB::transaction(function () use ($queue, $data): Message {
            $delaySeconds = (int) ($data['delay_seconds'] ?? 0);
            $availableAt = $delaySeconds > 0 ? now()->addSeconds($delaySeconds) : now();
            $status = $delaySeconds > 0 ? MessageStatuses::DELAYED : MessageStatuses::PENDING;

            $message = $queue->messages()->create([
                'payload' => $data['payload'],
                'priority' => $data['priority'] ?? 'normal',
                'status' => $status,
                'attempts' => 0,
                'max_attempts' => $data['max_attempts'] ?? 3,
                'available_at' => $availableAt,
            ]);

            if ($status === MessageStatuses::DELAYED) {
                $this->redisBroker->enqueueDelayed($queue->name, $message->id, $availableAt-
>timestamp);
            } else {
                $this->redisBroker->enqueueReady($queue->name, $message->id, $message->priority);
            }

            event(new BrokerStateUpdated($queue->name));

            return $message;
        });
    }
}
...

---

## 8. MessageConsumeService - выдача сообщения consumer-у

Файл: `app/Services/Broker/MessageConsumeService.php`

```php
<?php

namespace App\Services\Broker;

use App\Enum\MessageStatuses;
use App\Enum\QueueStatuses;
use App\Events\BrokerStateUpdated;
use App\Models\Message;
use App\Models\Queue;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Str;
use RuntimeException;

/**
 * MessageConsumeService выдаёт следующее доступное сообщение consumer-у.
 */
class MessageConsumeService
{
 public function __construct(

```

```

 private readonly RedisBroker $redisBroker,
 private readonly MessageRecoveryService $recoveryService,
) {}

 /**
 * Выдаёт сообщение на обработку.
 *
 * Алгоритм:
 * 1. Проверить, что очередь активна.
 * 2. Выполнить recovery delayed/expired сообщений.
 * 3. Вычислить reserved_until по visibility_timeout.
 * 4. Забрать id сообщения из Redis через consumeNext.
 * 5. В транзакции заблокировать строку сообщения в БД.
 * 6. Проверить, что сообщение всё ещё pending/failed.
 * 7. Сгенерировать ack_token.
 * 8. Перевести сообщение в processing.
 * 9. Сохранить token в Redis, а hash токена - в БД.
 * 10. Вернуть сообщение и открытый ack_token consumer-у.
 */
 public function consume(Queue $queue, array $data): ?array
 {
 if ($queue->status !== QueueStatuses::ACTIVE) {
 throw new RuntimeException('Queue is paused.');
 }

 $this->recoveryService->recoverQueue($queue);

 $visibilityTimeout = (int) ($data['visibility_timeout'] ?? 60);
 $reservedUntil = now()->addSeconds($visibilityTimeout);
 $messageId = $this->redisBroker->consumeNext($queue->name, $reservedUntil->timestamp);

 if ($messageId === null) {
 return null;
 }

 return DB::transaction(function () use ($queue, $data, $messageId, $reservedUntil, $visibilityTimeout): ?array {
 $message = Message::query()
 ->whereKey($messageId)
 ->where('queue_id', $queue->id)
 ->lockForUpdate()
 ->first();

 if (! $message || ! in_array($message->status, [MessageStatuses::PENDING, MessageStatuses::FAILED], false)) {
 $this->redisBroker->removeProcessing($queue->name, $messageId);

 return null;
 }

 $ackToken = Str::random(48);

 $message->update([
 'status' => MessageStatuses::PROCESSING,
 'attempts' => $message->attempts + 1,
 'consumer_id' => $data['consumer_id'],
 'reserved_until' => $reservedUntil,
 'ack_token_hash' => hash('sha256', $ackToken),
 'error' => null,
]);

 $this->redisBroker->setAckToken($message->id, $ackToken, $visibilityTimeout + 60);
 event(new BrokerStateUpdated($queue->name));

 return [
 'message' => $message->refresh(),
 'ack_token' => $ackToken,
];
 });
 }
}
...

```

## ## 9. MessageAckService - подтверждение или отклонение сообщения

Файл: `app/Services/Broker/MessageAckService.php`

```
```php
<?php

namespace App\Services\Broker;

use App\Enum\MessageStatuses;
use App\Events\BrokerStateUpdated;
use App\Models\Message;
use Illuminate\Support\Facades\DB;
use RuntimeException;

/**
 * MessageAckService обрабатывает ack и nack.
 *
 * ack = consumer успешно обработал сообщение.
 * nack = consumer не смог обработать сообщение.
 */
class MessageAckService
{
    public function __construct(private readonly RedisBroker $redisBroker) {}

    /**
     * Подтверждает успешную обработку.
     *
     * Метод сначала проверяет ack_token через withValidToken.
     * Если токен правильный:
     * - статус становится DONE;
     * - сохраняется result;
     * - reserved_until и ack_token_hash очищаются;
     * - completed_at фиксирует время завершения.
     */
    public function ack(Message $message, array $data): Message
    {
        return $this->withValidToken($message, $data['ack_token'], function (Message $message) use
($data): Message {
            $message->update([
                'status' => MessageStatuses::DONE,
                'result' => $data['result'] ?? null,
                'reserved_until' => null,
                'ack_token_hash' => null,
                'completed_at' => now(),
            ]);

            event(new BrokerStateUpdated($message->queue->name));

            return $message->refresh();
        });
    }

    /**
     * Сообщает об ошибке обработки.
     *
     * Если requeue=true и попытки ещё остались:
     * - сообщение получает статус FAILED;
     * - оно возвращается в ready-очередь Redis.
     *
     * Если попытки закончились или requeue=false:
     * - сообщение получает статус DEAD.
     */
    public function nack(Message $message, array $data): Message
    {
        return $this->withValidToken($message, $data['ack_token'], function (Message $message) use
($data): Message {
            $queue = $message->queue;
            $requeue = (bool) ($data['requeue'] ?? true);
            $hasAttemptsLeft = $message->attempts < $message->max_attempts;

            if ($requeue && $hasAttemptsLeft) {
                $message->update([

```

```

        'status' => MessageStatuses::FAILED,
        'error' => $data['error'] ?? null,
        'reserved_until' => null,
        'ack_token_hash' => null,
        'consumer_id' => null,
        'available_at' => now(),
    ]);

    $this->redisBroker->enqueueReady($queue->name, $message->id, $message->priority);
} else {
    $message->update([
        'status' => MessageStatuses::DEAD,
        'error' => $data['error'] ?? null,
        'reserved_until' => null,
        'ack_token_hash' => null,
        'completed_at' => now(),
    ]);
}

event(new BrokerStateUpdated($queue->name));

return $message->refresh();
});
}

/**
 * Общая проверка для ack и nack.
 *
 * Защита состоит из двух частей:
 * 1. Открытый token должен совпасть с Redis token.
 * 2. Hash открытого token должен совпасть с hash в БД.
 *
 * После успешной проверки сообщение убирается из Redis processing,
 * а token удаляется, чтобы повторный ack/nack был невозможен.
 */
private function withValidToken(Message $message, string $ackToken, callable $callback): Message
{
    return DB::transaction(function () use ($message, $ackToken, $callback): Message {
        $message = Message::query()->whereKey($message->id)->lockForUpdate()->firstOrFail();

        if ($message->status !== MessageStatuses::PROCESSING) {
            throw new RuntimeException('Message is not processing.');
        }

        $redisToken = $this->redisBroker->getAckToken($message->id);

        $isValid = $redisToken !== null
            && hash_equals($redisToken, $ackToken)
            && hash_equals((string) $message->ack_token_hash, hash('sha256', $ackToken));

        if (! $isValid) {
            throw new RuntimeException('Invalid or expired ack token.');
        }

        $this->redisBroker->removeProcessing($message->queue->name, $message->id);
        $this->redisBroker->deleteAckToken($message->id);

        return $callback($message);
    });
}
}

```

10. MessageRecoveryService - восстановление delayed и зависших processing

Файл: `app/Services/Broker/MessageRecoveryService.php`

```

` ``php
<?php

```

```

namespace App\Services\Broker;

```

```

use App\Enum\MessageStatuses;
use App\Events\BrokerStateUpdated;
use App\Models\Message;
use App\Models\Queue;
use Carbon\CarbonInterface;

/**
 * MessageRecoveryService приводит Redis и БД в актуальное состояние.
 */
class MessageRecoveryService
{
    public function __construct(private readonly RedisBroker $redisBroker) {}

    /**
     * Восстанавливает одну очередь:
     * - delayed сообщения, срок которых наступил;
     * - processing сообщения, у которых истёк visibility timeout.
     */
    public function recoverQueue(Queue $queue, int $limit = 100): array
    {
        $now = now();
        $timestamp = $now->timestamp;

        $delayed = $this->recoverDelayed($queue, $timestamp, $limit);
        $expired = $this->recoverExpiredProcessing($queue, $now, $limit);

        if ($delayed + $expired > 0) {
            event(new BrokerStateUpdated($queue->name));
        }

        return [
            'delayed' => $delayed,
            'expired' => $expired,
            'total' => $delayed + $expired,
        ];
    }

    /**
     * Запускает recovery для всех очередей.
     */
    public function recoverAll(int $limit = 100): array
    {
        $summary = ['delayed' => 0, 'expired' => 0, 'total' => 0];

        Queue::query()->each(function (Queue $queue) use (&$summary, $limit): void {
            $queueSummary = $this->recoverQueue($queue, $limit);

            $summary['delayed'] += $queueSummary['delayed'];
            $summary['expired'] += $queueSummary['expired'];
            $summary['total'] += $queueSummary['total'];
        });

        return $summary;
    }

    /**
     * Переносит delayed-сообщения в ready.
     *
     * Redis sorted set хранит message_id со score = timestamp.
     * Если score <= now, сообщение можно отдавать consumer-ам.
     */
    private function recoverDelayed(Queue $queue, int $timestamp, int $limit): int
    {
        $messageIds = $this->redisBroker->dueDelayed($queue->name, $timestamp, $limit);
        $count = 0;

        foreach ($messageIds as $messageId) {
            $message = Message::query()->whereKey($messageId)->where('status', MessageStatuses::DE-
LAYED)->first();
            $this->redisBroker->removeDelayed($queue->name, $messageId);

            if (!$message) {
                continue;
            }
        }
    }
}

```

```

        $message->update([
            'status' => MessageStatuses::PENDING,
            'available_at' => now(),
        ]);

        $this->redisBroker->enqueueReady($queue->name, $message->id, $message->priority);
        $count++;
    }

    return $count;
}

/**
 * Обрабатывает сообщения, которые были выданы consumer-у,
 * но consumer не прислал ack/nack до reserved_until.
 *
 * Если попытки закончились - DEAD.
 * Если попытки остались - снова PENDING и возвращаем в Redis ready.
 */
private function recoverExpiredProcessing(Queue $queue, CarbonInterface $now, int $limit): int
{
    $messageIds = $this->redisBroker->expiredProcessing($queue->name, $now->timestamp, $limit);
    $count = 0;

    foreach ($messageIds as $messageId) {
        $message = Message::query()->whereKey($messageId)->where('status', MessageStatuses::PRO-
CESSING)->first();

        $this->redisBroker->removeProcessing($queue->name, $messageId);
        $this->redisBroker->deleteAckToken($messageId);

        if (!$message) {
            continue;
        }

        if ($message->attempts >= $message->max_attempts) {
            $message->update([
                'status' => MessageStatuses::DEAD,
                'reserved_until' => null,
                'ack_token_hash' => null,
                'consumer_id' => null,
                'error' => $message->error ?: 'Visibility timeout expired after max attempts.',
                'completed_at' => now(),
            ]);
        } else {
            $message->update([
                'status' => MessageStatuses::PENDING,
                'reserved_until' => null,
                'ack_token_hash' => null,
                'consumer_id' => null,
                'available_at' => now(),
            ]);

            $this->redisBroker->enqueueReady($queue->name, $message->id, $message->priority);
        }

        $count++;
    }

    return $count;
}
}
...

---

## 11. Controllers - HTTP API

### QueueController

Файл: `app/Http/Controllers/QueueController.php`

```php

```



```

<?php

namespace App\Http\Controllers;

use App\Http\Requests\CreateQueueRequest;
use App\Http\Requests\UpdateQueueRequest;
use App\Http\Resources\QueueResource;
use App\Models\Queue;
use App\Services\Broker\QueueService;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use InvalidArgumentException;

/**
 * QueueController принимает HTTP-запросы по очередям
 * и передаёт работу в QueueService.
 */
class QueueController extends Controller
{
 public function __construct(private readonly QueueService $queueService) {}

 /**
 * GET /api/queues
 * Возвращает список очередей.
 */
 public function index()
 {
 return QueueResource::collection($this->queueService->list());
 }

 /**
 * POST /api/queues
 * Создаёт новую очередь после валидации CreateQueueRequest.
 */
 public function store(CreateQueueRequest $request): QueueResource
 {
 return new QueueResource($this->queueService->create($request->validated()));
 }

 /**
 * GET /api/queues/{queue:name}
 * Показывает одну очередь.
 *
 * Laravel автоматически ищет Queue по полю name из route model binding.
 */
 public function show(Queue $queue): QueueResource
 {
 return new QueueResource($queue);
 }

 /**
 * PATCH /api/queues/{queue:name}
 * Обновляет статус очереди.
 */
 public function update(UpdateQueueRequest $request, Queue $queue): QueueResource
 {
 return new QueueResource($this->queueService->update($queue, $request->validated()));
 }

 /**
 * DELETE /api/queues/{queue:name}
 * Удаляет очередь.
 *
 * Если очередь не пустая и force не передан,
 * QueueService выбросит InvalidArgumentException.
 */
 public function destroy(Request $request, Queue $queue): JsonResponse
 {
 try {
 $this->queueService->delete($queue, $request->boolean('force'));
 } catch (InvalidArgumentException $exception) {
 return response()->json(['message' => $exception->getMessage()], 422);
 }
 }
}

```

```

 return response()->json(status: 204);
 }
}
...

MessageController

Файл: `app/Http/Controllers/MessageController.php`

```php
<?php

namespace App\Http\Controllers;

use App\Http\Requests\AckMessageRequest;
use App\Http\Requests\ConsumeMessageRequest;
use App\Http\Requests\NackMessageRequest;
use App\Http\Requests\PublishMessageRequest;
use App\Http\Resources\MessageResource;
use App\Models\Message;
use App\Models\Queue;
use App\Services\Broker\MessageAckService;
use App\Services\Broker\MessageConsumeService;
use App\Services\Broker\MessagePublishService;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use RuntimeException;

/**
 * MessageController управляет сообщениями:
 * - список;
 * - публикация;
 * - consume;
 * - ack;
 * - nack.
 */
class MessageController extends Controller
{
    /**
     * GET /api/queues/{queue:name}/messages
     * Возвращает сообщения очереди с фильтрами status/priority.
     */
    public function index(Request $request, Queue $queue)
    {
        $messages = $queue->messages()
            ->when($request->query('status'), fn ($query, string $status) => $query->where('status',
            $status))
            ->when($request->query('priority'), fn ($query, string $priority) => $query->where('pri-
            ority', $priority))
            ->latest()
            ->paginate((int) $request->query('per_page', 25));

        return MessageResource::collection($messages);
    }

    /**
     * POST /api/queues/{queue:name}/messages
     * Публикует новое сообщение в очередь.
     */
    public function store(PublishMessageRequest $request, Queue $queue, MessagePublishService $pub-
    lishService): MessageResource
    {
        return new MessageResource($publishService->publish($queue, $request->validated()));
    }

    /**
     * POST /api/queues/{queue:name}/messages/consume
     * Выдаёт consumer-у следующее доступное сообщение.
     */
    public function consume(ConsumeMessageRequest $request, Queue $queue, MessageConsumeService
    $consumeService): JsonResponse
    {
        try {
            $result = $consumeService->consume($queue, $request->validated());

```

```

    } catch (RuntimeException $exception) {
        return response()->json(['message' => $exception->getMessage()], 409);
    }

    if ($result === null) {
        return response()->json([
            'data' => [
                'message' => null,
                'ack_token' => null,
                'empty' => true,
            ],
            'message' => 'No messages available.',
        ], 200);
    }

    return response()->json([
        'data' => [
            'message' => (new MessageResource($result['message']))->toArray($request),
            'ack_token' => $result['ack_token'],
            'empty' => false,
        ],
    ]);
}

/**
 * POST /api/messages/{message}/ack
 * Подтверждает успешную обработку сообщения.
 */
public function ack(AckMessageRequest $request, Message $message, MessageAckService $ackService): MessageResource|JsonResponse
{
    try {
        return new MessageResource($ackService->ack($message, $request->validated()));
    } catch (RuntimeException $exception) {
        return response()->json(['message' => $exception->getMessage()], 409);
    }
}

/**
 * POST /api/messages/{message}/nack
 * Сообщает об ошибке обработки сообщения.
 */
public function nack(NackMessageRequest $request, Message $message, MessageAckService $ackService): MessageResource|JsonResponse
{
    try {
        return new MessageResource($ackService->nack($message, $request->validated()));
    } catch (RuntimeException $exception) {
        return response()->json(['message' => $exception->getMessage()], 409);
    }
}
}
...

```

BrokerController

Файл: `app/Http/Controllers/BrokerController.php`

```

` ` ` php
<?php

namespace App\Http\Controllers;

use App\Http\Resources\MessageResource;
use App\Http\Resources\QueueResource;
use App\Models\Queue;
use App\Services\Broker\MessageRecoveryService;
use App\Services\Broker\QueueService;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

/**
 * BrokerController содержит общие endpoints брокера:
 * recovery, stats, dashboard.

```

```

*/
class BrokerController extends Controller
{
    /**
     * POST /api/broker/recover
     * Принудительно запускает восстановление всех очередей.
     */
    public function recover(MessageRecoveryService $recoveryService): JsonResponse
    {
        return response()->json(['data' => $recoveryService->recoverAll()]);
    }

    /**
     * GET /api/broker/stats
     * Возвращает общую статистику брокера.
     */
    public function stats(QueueService $queueService): JsonResponse
    {
        return response()->json(['data' => $queueService->stats()]);
    }

    /**
     * GET /api/broker/dashboard
     * Возвращает данные для dashboard:
     * - список очередей;
     * - статистику;
     * - последние сообщения выбранной очереди.
     */
    public function dashboard(Request $request, QueueService $queueService): JsonResponse
    {
        $queues = $queueService->list();
        $queueName = $request->query('queue') ? $queues->first()->name;
        $queue = $queueName ? Queue::query()->where('name', $queueName)->first() : null;

        $messages = $queue
            ? $queue->messages()
                ->when($request->query('status'), fn ($query, string $status) => $query->where('status', $status))
                ->when($request->query('priority'), fn ($query, string $priority) => $query->where('priority', $priority))
                ->latest()
                ->limit(25)
                ->get()
            : collect();

        return response()->json([
            'data' => [
                'queues' => QueueResource::collection($queues)->resolve($request),
                'stats' => $queueService->stats(),
                'messages' => MessageResource::collection($messages)->resolve($request),
            ],
        ]);
    }
}

```

12. Form Requests - валидация входных данных

Файлы: `app/Http/Requests/\*.php`

```

` ` ` php
<?php

/**
 * CreateQueueRequest:
 * - поле обязателен;
 * - минимум 3 символа;
 * - максимум 255;
 * - разрешены буквы, цифры, _, ., -;
 * - имя должно быть уникальным в таблице queues.
 */

```

```

/**
 * PublishMessageRequest:
 * - payload обязателен и должен быть массивом;
 * - priority должен быть high/normal/low;
 * - delay_seconds от 0 до 604800 секунд;
 * - max_attempts от 1 до 25.
 */

/**
 * ConsumeMessageRequest:
 * - consumer_id обязателен;
 * - visibility_timeout задаёт время, за которое consumer должен сделать ack/nack.
 */

/**
 * AckMessageRequest:
 * - ack_token обязателен;
 * - result можно передать как массив.
 */

/**
 * NackMessageRequest:
 * - ack_token обязателен;
 * - error можно передать строкой;
 * - requeue показывает, возвращать ли сообщение в очередь.
 */

/**
 * UpdateQueueRequest:
 * - можно передать status=active/paused;
 * - или paused=true/false.
 */
` ``

```

13. Resources – формат JSON-ответов

Файлы: ``app/Http/Resources/*.php``

```

` `` php
<?php

/**
 * MessageResource превращает Message-модель в JSON:
 * - отдаёт id, queue_id, queue_name;
 * - отдаёт payload, status, priority;
 * - отдаёт attempts/max_attempts;
 * - отдаёт даты в ISO-формате;
 * - отдаёт result/error.
 */

/**
 * QueueResource превращает Queue-модель в JSON:
 * - id;
 * - name;
 * - status;
 * - stats;
 * - created_at/updated_at.
 *
 * Внутри дополнительно вызывает QueueService::statsForQueue,
 * чтобы сразу показать статистику конкретной очереди.
 */

/**
 * QueueStatsResource нормализует статистику:
 * - если какого-то статуса нет, возвращает 0;
 * - отдельно показывает состояние Redis: ready/delayed/processing.
 */
` ``

```

14. Event BrokerStateUpdated

Файл: `app/Events/BrokerStateUpdated.php`

```
```php
<?php

namespace App\Events;

use App\Http\Resources\MessageResource;
use App\Http\Resources\QueueResource;
use App\Models\Queue;
use App\Services\Broker\QueueService;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Http\Request;
use Illuminate\Queue\SerializesModels;

/**
 * BrokerStateUpdated отправляется, когда состояние брокера изменилось:
 * - создана/удалена очередь;
 * - опубликовано сообщение;
 * - сообщение взято в processing;
 * - сообщение ack/nack;
 * - recovery что-то изменил.
 *
 * ShouldBroadcastNow означает, что событие отправляется сразу.
 */
class BrokerStateUpdated implements ShouldBroadcastNow
{
 use Dispatchable;
 use InteractsWithSockets;
 use SerializesModels;

 public function __construct(
 public readonly string $queueName,
) {}

 /**
 * Канал broadcast.
 */
 public function broadcastOn(): Channel
 {
 return new Channel('broker');
 }

 /**
 * Имя события на frontend.
 */
 public function broadcastAs(): string
 {
 return 'broker.state.updated';
 }

 /**
 * Payload события.
 *
 * Здесь собирается snapshot:
 * - очереди;
 * - статистика;
 * - последние 25 сообщений выбранной очереди.
 */
 public function broadcastWith(): array
 {
 $request = Request::create('/broadcasting/broker-state');
 $queueService = app(QueueService::class);

 $queues = $queueService->list();
 $queue = Queue::query()->where('name', $this->queueName)->first() ?? $queues->first();

 $messages = $queue
 ? $queue->messages()->latest()->limit(25)->get()
 : collect();
 }
}
```

```

 return [
 'queue' => $this->queueName,
 'snapshot' => [
 'queues' => QueueResource::collection($queues->resolve($request),
 'stats' => $queueService->stats(),
 'messages' => MessageResource::collection($messages->resolve($request),
],
];
 }
}
...

15. Routes API

Файл: `routes/api.php`

```php
<?php

use App\Http\Controllers\BrokerController;
use App\Http\Controllers\MessageController;
use App\Http\Controllers\QueueController;
use Illuminate\Support\Facades\Route;

/**
 * Очереди.
 */
Route::get('/queues', [QueueController::class, 'index']);
Route::post('/queues', [QueueController::class, 'store']);
Route::get('/queues/{queue:name}', [QueueController::class, 'show']);
Route::patch('/queues/{queue:name}', [QueueController::class, 'update']);
Route::delete('/queues/{queue:name}', [QueueController::class, 'destroy']);

/**
 * Сообщения внутри очередей.
 */
Route::get('/queues/{queue:name}/messages', [MessageController::class, 'index']);
Route::post('/queues/{queue:name}/messages', [MessageController::class, 'store']);
Route::post('/queues/{queue:name}/messages/consume', [MessageController::class, 'consume']);

/**
 * Ack/nack работают по id сообщения.
 */
Route::post('/messages/{message}/ack', [MessageController::class, 'ack']);
Route::post('/messages/{message}/nack', [MessageController::class, 'nack']);

/**
 * Общие endpoints брокера.
 */
Route::post('/broker/recover', [BrokerController::class, 'recover']);
Route::get('/broker/stats', [BrokerController::class, 'stats']);
Route::get('/broker/dashboard', [BrokerController::class, 'dashboard']);
```

```

## ## 16. Migrations – структура базы данных

Файлы: `database/migrations/\*.php`

```

```php
<?php

/**
 * create_queues_table:
 *
 * Создаёт таблицу queues:
 * - id;
 * - name, уникальное имя очереди;
 * - status, active/paused;
 * - timestamps.
 */

```

```

*/

/**
 * create_messages_table:
 *
 * Создаёт таблицу messages:
 * - queue_id связан с queues.id;
 * - payload хранит JSON-сообщение;
 * - status хранит состояние сообщения;
 * - priority хранит high/normal/low;
 * - attempts/max_attempts контролируют retry;
 * - available_at нужно для delayed сообщений;
 * - reserved_until нужно для visibility timeout;
 * - consumer_id показывает, кто взял сообщение;
 * - ack_token_hash хранит hash токена;
 * - result хранит результат успешной обработки;
 * - error хранит ошибку;
 * - completed_at фиксирует финальное завершение.
 */

/**
 * add_status_column_to_queues_table:
 *
 * Защитная миграция: добавляет status в queues,
 * только если такого столбца ещё нет.
 */

/**
 * normalize_queue_status_values:
 *
 * Нормализует старые значения статуса:
 * - 1/true/t -> active;
 * - 0/false/f -> paused.
 *
 * Для PostgreSQL дополнительно обновляет check constraint.
 */

/**
 * create_personal_access_tokens_table:
 *
 * Стандартная таблица Laravel Sanctum для API-токенов.
 */
```

```

---

## ## 17. Полный жизненный цикл сообщения

```

```text
1. Клиент создаёт очередь:
   POST /api/queues

2. Клиент публикует сообщение:
   POST /api/queues/{queue}/messages

3. Если delay_seconds = 0:
   БД: status = pending
   Redis: ready list

4. Если delay_seconds > 0:
   БД: status = delayed
   Redis: delayed sorted set

5. Consumer забирает сообщение:
   POST /api/queues/{queue}/messages/consume

6. Сообщение становится processing:
   БД: status = processing
   Redis: processing sorted set
   Redis: ack token с TTL

7. Если consumer успешно обработал:
   POST /api/messages/{id}/ack
   БД: status = done

```



```

8. Если consumer обработал с ошибкой:
   POST /api/messages/{id}/nack

9. Если попытки остались:
   БД: status = failed
   Redis: ready list

10. Если попытки закончились:
   БД: status = dead

11. Если consumer пропал:
   recovery найдёт expired processing и вернёт сообщение в pending
   или переведёт в dead.
...

---
```

18. Главная идея архитектуры

Backend разделён на слои:

```

```text
HTTP Request
 ↓
FormRequest validation
 ↓
Controller
 ↓
Service
 ↓
Model + RedisBroker
 ↓
Database + Redis
 ↓
Resource
 ↓
JSON Response
```
```

Такой подход удобен, потому что:

1. *Controller* не содержит сложную бизнес-логику.
2. *Service* можно тестировать отдельно.
3. *RedisBroker* изолирует работу с *Redis*.
4. *Model* отвечает только за структуру данных и связи.
5. *Resource* контролирует формат ответа *API*.

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы